REVISION LETTER OF PAPER 560 → 1539

Dear Meta Reviewer and Referees:

We would like to thank the Referees for insightful comments and for supporting this work! We have substantially revised the paper following the Referees' suggestions. For your convenience, changes for addressing the concerns of R1, R2 and R3 are color-coded in blue, purple and orange, respectively.

To accommodate the suggested changes, we added some of the new experiments to the appendix of [4].

Below please find our responses to the reviews.

Response to the comments of Meta Reviewer

[Meta summary & recommendation] *The paper studies an important and timely problem in latency-critical GraphRAG serving, and the reviewers acknowledged the clear motivation, novel design, and promising empirical results. However, important concerns remain regarding evaluation, robustness under difficult queries and bursty workloads, deployment practicality, metric design, and clarity of several technical details. After considering the author feedback, all reviewers agreed that these concerns can be addressed during revision, and I therefore recommend Major Revision.*

[A] Thank you for the helpful meta-review and for recommending a major revision. We have substantially revised the paper to address the concerns regarding evaluation, robustness under difficult and bursty workloads, deployment practicality, metric design, and technical clarity. More specifically, we have:

∘ Improved technical clarity and metric design (addressing R1.O1–R1.O3, R1.Q1, R2.O1–R2.O2, R2.O4, and R3.O1–R3.O2) by defining the query-induced subgraph, candidate evidence units, K_{diff}, and experimental notations; clarifying adaptive budgeting, the Phase-3-to-Phase-2 backpressure loop, scheduling urgency, and the control boundary under unpredictable decode lengths; adding the retrieval-ambiguity guardrail; replacing the Pareto-optimality claim with a precise empirical claim; and complementing hard-deadline QA-Goodput with soft time utility;

∘ Substantially expanded and disentangled the evaluation (addressing R1.O3, R2.O1, R2.O4–R2.O5, R3.O1–R3.O2, and R3.O4–R3.O6) by adding quality–latency and soft-utility comparisons, the AdaptiveRAG baseline, false-easy/difficulty-estimator audits, controlled Phase-1 candidate-source and backpressure-scope studies, serving-strategy and phase ablations, energy/ITL measurements, and simultaneous load-and-difficulty burst and recovery tests; and

∘ Strengthened deployment practicality, fairness, and reproducibility (addressing R2.O2–R2.O3 and R3.O3) by documenting one-time model–hardware calibration and sensitivity; graph-construction agnosticism subject to the required index interface and micro-batch/incremental maintenance; the non-preemptive decoding boundary and complementary decode/admission mitigations; and matched baseline protocols, complete configurations, online-cost accounting, and release artifacts.

Response to the comments of Referee #1

[R1.O1] *More clarification is needed in the method presentation. For example, in structure-semantic collaborative retrieval in Section 4.2, a candidate subgraph can be defined clearer. In Section 4.2, the proposed method seems to combine selected documents, chunks, and entities into a single set C_q. The paper should clarify the definition of each element in C_q. Likewise, Section 4.3.3 can clarify the definition of "d" in "each $d \in C_q$." Also, the definition of "K_{diff}" in Section 4.3.3 seems to be missing.*

[A] Thank you for this helpful comment! We have carefully checked the paper and improved presentation quality, especially notation consistency and object definitions. Specifically, we have made the following clarifications in Sections 4.2, 4.3.2, and 4.3.3 (pp. 6–8).

(1) In Section 4.2 (p. 6), we have defined the query-induced subgraph $\mathcal{G}_q$ as the subgraph induced by Hop-1 semantic anchoring and Hop-2 graph expansion, including the anchor chunks, their document/entity nodes, expanded entities, and traversed inclusion/relational edges. We have also clarified that $C_q = C_q^{\text{chunk}} \cup C_q^{\text{doc}} \cup C_q^{\text{graph}}$ contains three sources of *chunk-level evidence*: semantically retrieved chunks, source chunks recovered through their parent documents, and source chunks mapped from graph-expanded entities.

(2) In Section 4.3.3 (p. 8), we have clarified that each $d \in C_q$ denotes such a chunk-level evidence unit.

(3) In Section 4.3.2 (p. 7), we have defined $K_{\text{diff}}(q)$ as the quality-first budget determined by query difficulty before backpressure, and clarified its use in context assembly in Section 4.3.3.

[R1.O2] *Several notations in the experiments should be clearer. For example, the paper should provide the definitions of Linear, Fixed, Threshold in Section 5.4. The definition of $d_{\max}$ is missing in Section 5.4.*

[A] Thank you for catching this! We have carefully checked and improved the presentation of experimental notations. Specifically, we have made the following clarifications in Section 5.4 (p. 12).

(1) In the decay-shape ablation (p. 12), we now define all four tested strategies: (a) *Exponential* uses the load-dependent decay in Eq. (8); (b) *Linear* decreases $K_{\text{eff}}(t)$ linearly with the normalized excess utilization $(u(t) - \theta_{\text{bp}})/(1 - \theta_{\text{bp}})$; (c) *Fixed* sets $K_{\text{eff}}(t) = K_{\text{bp}}$ to a constant reduced ceiling once congestion is triggered; and (d) *Threshold* directly sets $K_{\text{eff}}(t) = K_{\min}$ whenever $u(t) > \theta_{\text{bp}}$.

(2) In the topology-feature ablation (p. 12), we now define $d_{\max}$ as the maximum node degree in the query-induced subgraph $\mathcal{G}_q$.

[R1.O3] *How can the paper ensure that the proposed method operates near its Pareto-optimal point via adaptive budgeting and backpressure? This claim can be empirically validated by comparing this method with baseline methods in terms of the quality-latency tradeoff.*

[A] Thank you for pointing this out! We agree that the phrase "near its Pareto-optimal point" is too strong without an explicit Pareto-frontier analysis. We have made the following revisions in the revised draft.

(1) In the initial submission, Section 5.6 stated that DynaGraph operates near its Pareto-optimal point. We have moved the corresponding model-scalability discussion to Appendix E.6 in [4] and revised the claim: DynaGraph preserves the quality–latency tradeoff while operating in a favorable empirical region, rather than claiming theoretical Pareto optimality.

(2) To support this claim, we have added an empirical SLO-aware quality–latency analysis in Section 5.2 (p. 11). As shown in the newly added Figure 6, DynaGraph lies on the empirical frontier in the ($\mathcal{J}$@500, median TTFT) plane, outperforms the evaluated

GraphRAG baselines, and matches or slightly improves upon the best fixed-budget operating point without requiring the budget to be selected prior to deployment.

[R1.Q1] *Several mistakes should be fixed. (1) A closing parenthesis is missing in the problem input in Section 2.2: "(from witch a heterogeneous graph . . ."; (2) in Section 5.5, Kmin should be $K_{\min}$; and (3) θbp should be θ_{bp}.*

[A] Thank you for catching these! We have carefully proofread the paper and fixed these typos and notation issues (*e.g.,* pp. 3 and 13).

Many thanks for recognizing the novelty of our work, for your support, and for your careful reading and constructive suggestions!

Response to the comments of Referee #2

[R2.O1] *A key technical risk lies in the accuracy of the cost-estimation layer. Because the difficulty score (S_{diff}) is a coarse classifier based purely on structural features, it may misclassify queries on sparse graphs that still require complex textual reasoning as "easy." This may lead to over-compressing the context budget, risking severe quality drop-offs that structural metrics cannot predict. I was wondering if there is a way to address this issue.*

[A] Thank you for this insightful comment! We agree that this risk is possible: a sparse query-induced graph may still require complex textual reasoning, causing a purely structural estimator to underestimate difficulty and over-compress the context budget. We have made the following revisions in Sections 4.3.2 and 5.4 (pp. 7, 8 and 12).

(1) In Section 4.3.2 (pp. 7–8), we have added a discussion under *Retrieval-ambiguity guardrail* on how DynaGraph can be extended to handle such cases. This discussion is also cued in both the opening paragraph of Section 4.3.2 and the note under *Cost-aware difficulty estimator*. That is, although $S_{\text{diff}}(q)$ provides a zero-cost topology prior, it can yield false "easy" cases when the query-induced graph is sparse but the retrieved textual evidence is ambiguous or conflicting. In such cases, topology-only scoring may over-compress the context budget and overlook quality risks not reflected in structural metrics. To reduce this risk, DynaGraph can optionally use the dispersion of the top retrieval scores in Phase 1 as a lightweight check before budget allocation. When topology predicts an easy query but retrieval evidence is uncertain, the controller can conservatively increase the assigned budget, capped by the current budget ceiling. This provides a lightweight optional check for topology-induced false "easy" cases without changing confident topology-easy queries or requiring additional model inference.

(2) In Section 5.4 (p. 12), we have added a difficulty-estimator audit targeting false "easy" cases, comparing against two diagnostic methods: RAG-CSM [15] and Adaptive-K [47]. As shown in Table 5, even without the guardrail, S_{diff} outperforms RAG-CSM (0.260/0.656/0.326 vs. 0.254/0.638/0.307 in Spearman ρ/AUROC/AUPRC) with negligible latency (< 0.001 ms vs. 25.33 ms), and also substantially outperforms the score-ratio heuristic of Adaptive-K (0.013/0.457/0.207). With the retrieval-ambiguity guardrail, combining retrieval-score dispersion with topology further improves the audit to 0.375/0.695/0.391, with only 0.009 ms selector latency. Moreover, the same-stack online comparison (Appendix E.7 in [4]) shows that structural S_{diff} improves $\mathcal{J}$@500 from 75.35% for live RAG-CSM to 79.27% and reduces p95 TTFT from 513.6 to 317.7 ms, with no statistically significant difference in request F1. These results show that topology-based estimation is competitive with alternative estimators, while the matched-budget audit shows that retrieval-score dispersion improves estimator–oracle alignment in false "easy" cases. Accordingly, we attribute the online serving improvement to structural S_{diff} and present the retrieval-ambiguity guardrail as a low-overhead optional check supported by the diagnostic audit.

[R2.O2] *DynaGraph manages prefill length but leaves the decoding execution loop unmanaged because output length is unpredictable (depends on the model's inner generation paths). This may be risky as missing context can trigger longer generation loops, increasing decode costs. If multiple urgent queries experience this unpredicted elongation, the non-preemptive GPU queue could suffer resource starvation or SLO failures that the upstream compressor cannot prevent.*

[A] Thank you for this constructive comment! We agree that decode-length variability is an important serving risk for non-preemptive LLM serving [52]: output length is inherently variable, making decode cost difficult to predict at admission time.

We have made the following revisions in Section 4.4 (p. 9), and in Appendices E.3 and E.10 of [4].

(1) In Section 4.4 (p. 9), we have clarified DynaGraph's control boundary. The feedback from Phase 3 acts only on the evidence budget: retriever expansion and retrieved candidates remain fixed during serving, while the realized prefill cost is determined by K_q. Thus, DynaGraph does not predict or preempt an active decode. Instead, long generations can delay later prefills and indirectly increase the pending-prefill backlog captured by $u(t)$, causing subsequent admissions to receive tighter budgets.

(2) In Appendix E.10 of [4], we have added a load-and-difficulty burst experiment to evaluate queueing and strict-SLO behavior under simultaneous arrival-rate and evidence-demand pressure; see also [R3.O2]. Appendix E.3 of [4] further isolates how missing gold evidence prolongs generation and propagates to queueing and strict-SLO degradation. These results show that, although DynaGraph cannot eliminate decode-length variability, its admission-time budget adaptation can indirectly limit the resulting queue pressure.

(3) In Section 4.4 (p. 9), we have discussed complementary scheduling-level mitigations. Preemptive decode mechanisms (*e.g.,* FastServe [52]) and output-length-aware admission caps can directly manage active long generations, whereas DynaGraph controls admission-time evidence budgets and scheduling priority. We leave their full integration to future work.

[R2.O3] *Practical production adoption of DynaGraph seems to introduce two main operational hurdles. First, it requires manual calibration of difficulty weights and hardware-specific throughput thresholds. Second, it depends on a clean, offline-constructed graph index; keeping this graph synchronized with real-time, streaming enterprise data would be a significant challenge. It would be good to have guidance in the paper on how to address these issues.*

[A] Thanks for raising this important deployment concern! We have made the following revisions in Sections 4.2, 4.4 and 5.5 (pp. 6, 9 and 13), and in Appendix E.8 of [4].

(1) In Section 4.4 (p. 9), we have clarified that DynaGraph requires

only one-time calibration: μ_p is profiled once per model–hardware pair, $\mathbf{w}$ is selected from query-grouped calibration data and fixed during serving, and θ_{bp} is a dimensionless utilization threshold. No workload-specific retuning occurs on the serving path.

In Appendix E.8 in [4], we evaluate calibration-sample sufficiency over 20 query-grouped nested-validation splits (fixed 300-query test fold; $N = 25$–700) and coefficient sensitivity on a matched 5-QPS schedule. Agreement with the $N = 700$ reference reaches 0.958 at $N = 200$ and 0.969 at $N = 300$, while the deployed, equal and historical post-hoc weights yield a request-F_1 range of only 0.56 percentage points and mean K of 13.46–13.76 (the post-hoc weights were diagnostic, not deployed). Moreover, K_{min} remains effective over $[5, 10]$, while $\mathcal{J}_\lambda$@500 varies by at most 1.4 percentage points over $\theta_{bp} \in [0.1, 0.9]$. These results show that calibration requires neither extensive labeled data nor sensitive or continuous retuning. We add a reference to this test briefly in Section 5.5 (p. 13).

(2) In Section 4.2 (p. 6), we have clarified that DynaGraph is agnostic to graph construction, provided that the index exposes the document/chunk/entity structure needed to derive Phase 1 topology signals. For domains lacking suitable general-purpose ontologies, we discuss domain-specific ontology extraction and validation for ontology-aware KG construction [12, 35], and industrial continuous KG platforms for index maintenance [23]. We also discuss micro-batch or incremental refresh of affected documents/entities while serving uses the latest consistent graph snapshot, and leave the co-design of ontology construction, graph maintenance, and SLO-aware serving to future work.

[R2.O4] *The custom QA-Goodput metric seems to inherently favor DynaGraph by assigning zero credit to queries that miss strict TTFT deadlines. Consequently, retrieval-heavy baselines score 0.0% simply by exceeding the time gates. While this reflects hard system constraints, a smoother, multi-tiered utility function would more objectively reveal the quality penalty paid when context budgets are compressed under peak load.*

[A] Thanks for this helpful suggestion! We have added the following in Sections 2.2 and 5.2 (pp. 3 and 11).

(1) In Section 2.2 (p. 3), we have introduced a soft time utility for settings where late responses retain partial value. It assigns full credit to responses meeting the deadline, linearly decaying credit to those arriving between τ_{q_i} and $2\tau_{q_i}$, and zero credit thereafter. We have also clarified that hard-deadline QA-Goodput and soft time utility are complementary, with their suitability depending on the deployment setting.

(2) In Section 5.2 (p. 11), we have strengthened the evaluation by reporting the newly introduced soft time utility in Table 3. As shown there, even when late responses receive partial credit, DynaGraph achieves the highest $\mathcal{J}^{soft}$@500 across all three datasets (74.6%/54.9%/47.6%), outperforming the strongest baselines by 26.7/14.1/18.2 percentage points, respectively. These results validate DynaGraph's advantage under both hard and soft latency utilities.

[R2.O5] *To further validate DynaGraph's capabilities, the evaluation can be enhanced to (a) track cost-per-token or energy efficiency to prove whether the CPU/GPU cycles spent on real-time graph parsing and continuous back pressure math actually save money compared to static chunking systems, (b) monitor Inter-Token Latency to ensure dynamic budget adjustments do not cause model's text generation to stutter due to resource reallocation, and (c) test the system under sudden traffic spikes to ensure the feedback loop stabilizes without oscillating wildly.*

[A] Thank you for the suggestion. We have added the suggested evaluations in Appendices E.9 and E.10 of [4] and cited them at the end of Section 5.5 (p. 13).

(1) *Energy efficiency.* In Appendix E.9 of [4], following prior LLM-inference energy-measurement studies [40, 57], we measure GPU energy per request and per 1K output tokens for DynaGraph and static-budget systems. Relative to static K=20, DynaGraph reduces these metrics by 24.7% and 15.6%, respectively, on the official 30-QPS trace. We also conduct a matched-budget experiment at 5 QPS: DynaGraph uses 9.35% less energy per request than static K=20, but 3.09% more than the approximately budget-matched static K=13. Thus, the reduction relative to K=20 primarily arises from shorter admitted contexts, while the matched-budget comparison does not isolate additional controller-specific savings. Overall, adaptive budgeting reduces total measured GPU energy relative to static K=20. These measurements exclude host energy and monetary cost.

(2) *ITL.* Also in Appendix E.9 of [4], we report token-level inter-token latency (ITL) distributions and threshold-exceedance rates for DynaGraph and static K=20. Although the 50- and 100-ms exceedance rates increase slightly, DynaGraph preserves median ITL, reduces p95/p99 ITL by 31.8%/34.9%, and reduces gaps above 200,ms by 32.6%. Thus, adaptive budgeting preserves typical streaming cadence while reducing severe tail stalls, although it does not improve every token-gap threshold.

(3) *Traffic spikes.* In Appendix E.10 of [4], we add a $5 \rightarrow 30 \rightarrow 5$-QPS load-and-difficulty burst with 70% hard-query arrivals at the peak and cite this study in Section 5.5 (p. 13). We track $u(t)$, $K_{eff}(t)$, and per-request TTFT, and report post-spike recovery diagnostics for DynaGraph-Fixed and DynaGraph-Exp under configuration-specific traces. On the primary burst trace, the controller contracts its effective budget during the spike and restores it afterward. Under the recovery traces, neither configuration exhibits an observed post-peak threshold recrossing or controller overshoot. Together, these results show that the feedback loop responds to sudden load changes and recovers without observed oscillation.

Many thanks for recognizing the novelty of our work, for your support, and for your constructive technical suggestions!

Response to the comments of Referee #3

[R3.O1] *It is not clear how the authors apply the backpressure to the retrieval phase. Does it regard the adaptation of K only?*

[A] Thank you for this helpful comment! We have added the following in Sections 4.1 and 5.4 (pp. 5 and 12).

(1) In Section 4.1 (p. 5), we have clarified that Phase 3 feeds congestion signals S_{sys} (*e.g.,* queue depth) back to Phase 2, closing the feedback loop for jointly optimizing the evidence budgets $\mathbf{K}$ and execution order π under load. The backward feedback acts only on K_q (and thus the prompt length L_q), without changing Phase 1 retrieval.

(2) In Section 5.4 (p. 12), we have added a sensitivity study that varies the static Phase 1 Hop-1 retrieval depth M while fixing the downstream controller, scheduler, corpus, reader and arrival trace.

As shown in Table 6, the default M=5 achieves the highest F1 and $\mathcal{J}$@500 among the tested settings. Increasing M offers a modest gain under the stricter 200-ms deadline, but reduces F1 and $\mathcal{J}$@500 and substantially increases the isolated Phase 1 cost; beyond M=20, it yields no further improvement in answer quality or QA-Goodput. These results support our design choice to keep Phase 1 retrieval fixed and apply backpressure only between Phases 2 and 3, where it directly adapts the downstream evidence budget K_q.

[R3.O2] *How does Dynagraph cope with bursty arrivals and difficult/budget-heavy queries at the same time?*

[A] Thank you for this insightful comment! We have made the following revisions in Sections 4.3.2, 4.4, 5.4 and 5.5 (pp. 7–9 and 12–13), and in Appendix E.10 of [4].

(1) In Section 4.3.2 (p. 7), we have clarified that, under congestion, Eq. (9) compresses the budget range while preserving the difficulty ordering: hard queries remain near its upper end rather than collapsing to $K_{\min}$. Moreover, in Section 4.4 (pp. 8–9), we have clarified that Phase 3 amplifies SLO urgency using a monotone function of the Phase 2 difficulty score, giving harder queries greater scheduling urgency rather than systematically deprioritizing them.

(2) In addition to the hard/easy separation test in Section 5.4 (p. 12), we have added a load-and-difficulty burst experiment in Appendix E.10 in [4] and cited it in Section 5.5 (p. 13). The experiment uses a $5 \rightarrow 30 \rightarrow 5$-QPS trace with 70% hard-query arrivals at the peak. For hard queries, the default exponential controller reduces p95 TTFT from 848 to 668 ms and improves $\mathcal{J}$@500 from 70.3% to 78.4%, with a 0.57-point F1 loss relative to static K=20. An aggressive constant-ceiling variant further reaches 435 ms and 86.9%, with a larger 1.58-point F1 loss. The retained hard/easy budgets show that exponential decay better preserves difficulty differentiation, whereas the constant ceiling compresses both strata more aggressively. These results illustrate the supported quality–latency trade-offs when high load coincides with difficult queries (see also our response to [R2.O5]).

[R3.O3] *Did the authors properly configure the competing tools for a fair comparison in experiments?*

[A] Thank you for raising this concern! We have made all baseline configurations fully verifiable. Within each matched comparison, all methods use the same LLM, hardware, workload generator, and request set. We use official implementations and settings where available, and document method-specific backend requirements and online-cost accounting. We detail each baseline's configuration in Appendix D of [4], including its retrieval depth, iteration count, context limit, serving backend, and any necessary adaptation. These details are available in the full version (https://anonymous.4open.science/r/DynaGraph-Releasee-0496).

[R3.O4] *In accord with Section 5.2. it would be very interesting to fix the serving layer and compare Phases 1 and 2 with retriever options.*

[A] Thank you for this helpful suggestion! We have added a controlled Phase 1 candidate-source study in Appendix E.1 of [4] and cited it in Section 5.4 (p. 11). We fix the serving layer, budget policy, reranker, reader, and average realized budget ($K \approx 10$), varying only the Phase 1 source among chunk, graph, chunk–document, and full-union configurations. This design isolates upstream retrieval evidence without confounding differences in indexes, retrieval budgets, or candidate formats.

As shown in Table 15 (in [4]), the full union achieves 0.773 F1 and 79.4% $\mathcal{J}$@500, versus 0.499/61.2%, 0.241/17.0%, and 0.510/59.3% for chunk-only, graph-only, and chunk–document retrieval, respectively. These results show that complementary document and graph evidence is critical to DynaGraph's quality–latency gains under a fixed serving pipeline. Together with RQ2, which fixes Phase 1 while varying the serving strategies, this study separates the contributions of retrieval and serving.

[R3.O5] *I would expect to see AdaptiveRAG in the experiments.*

[A] As suggested, we have added AdaptiveRAG [26] as a new baseline in Section 5.1 (p. 10) and compared it with DynaGraph in Section 5.2 (p. 11). As shown in Table 3, DynaGraph outperforms AdaptiveRAG by 3.3×–4.0× in $\mathcal{J}$@500. This is because, rather than relying on iterative retrieval–generation with multiple LLM calls per request as in AdaptiveRAG, DynaGraph coordinates evidence budgeting and scheduling before a single generation, substantially reducing serving costs under the shared load.

[R3.O6] *Overall, the experiments deserve further attention to clearly show the contribution of the approach.*

[A] Thank you for this comment! We have strengthened the evaluation through: quality–latency analysis (R1.O3); the AdaptiveRAG baseline (R3.O5); a Phase-1 candidate-source study (R3.O4); backpressure-scope analysis (R3.O1); false-easy/difficulty-estimator audits (R2.O1); soft-utility evaluation (R2.O4); energy/ITL measurements (R2.O5); burst/recovery tests (R2.O5/R3.O2); and serving-strategy/phase ablations (R3.O6). Clarified definitions (R1.O1–O2) and mechanisms (R3.O1–O2) further disentangle Phase-1 retrieval signals, Phase-2 adaptive budgeting, and Phase-3 SLO-aware scheduling, showing that their coordination yields DynaGraph's quality–latency gains.

Many thanks for recognizing the novelty of our work, for your support, and for your constructive suggestions on strengthening the experimental evaluation!

DynaGraph: Holistic Optimization for Latency-Critical GraphRAG Serving via Bidirectional Feedback

ABSTRACT

Graph Retrieval-Augmented Generation (GraphRAG) has emerged as a promising paradigm for complex multi-hop reasoning, bridging knowledge graphs with LLMs to answer questions requiring chained evidence. However, deploying GraphRAG in latency-critical serving systems faces a fundamental challenge—the *phase-silo design*: retrievers maximize recall without cost awareness, while serving engines optimize throughput without accounting for query difficulty. This mismatch leads to head-of-line blocking and SLO violations under concurrent and skewed workloads. To address this, we propose *DynaGraph*, a unified framework that coordinates retrieval signals with adaptive aggregation and scheduling through a closed-loop feedback design. DynaGraph (a) exposes graph-topology features as cost priors, (b) adapts the evidence budget under real-time backpressure, and (c) employs an SLO-aware scheduler to maximize a newly proposed *QA-Goodput* metric that balances answer quality and timeliness; it requires no LLM or retriever finetuning, and operates as lightweight middleware above modern LLM serving engines. Experiments on three multi-hop QA benchmarks across five LLM architectures show that DynaGraph improves QA-Goodput by 2.4×–3.3× and reduces median TTFT by 56%–64% over the strongest GraphRAG baseline, while achieving sub-linear latency scaling under 7× corpus growth. Code, artifacts and the full version are available online.[1]

1 INTRODUCTION

Online question answering (QA) is rapidly evolving from offline benchmarks to interactive, latency-critical deployments, where systems must answer user queries in real-time using large knowledge sources and language models [33]. Unlike offline evaluation, where latency is largely irrelevant, online QA must deliver responses within strict time budgets while preserving answer quality.

Consider an incident-response copilot used by site reliability engineers during a production outage [6]. An engineer may ask: *"Which customer-facing endpoints are affected by Service X's failure, and what mitigation steps were applied in similar past incidents?"* Answering such queries requires multi-hop reasoning over heterogeneous artifacts: service dependency graphs, runbooks, and incident summaries [13]. Crucially, the system must satisfy a strict *Time-To-First-Token* (TTFT) budget [32, 67], *i.e.,* the latency between issuing a request and receiving the first token of the response. In such high-stakes settings, answers must be not only correct but also delivered within a *Service Level Objective* (SLO) over TTFT (*e.g.,* TTFT ≤ 2 s for 99% of requests) [18]. A late (even correct) answer violates this SLO and breaks the human-in-the-loop cycle where engineers refine diagnostic queries based on prior responses [36, 49].

Example 1: We use an e-commerce customer-support scenario as RE-1: a user reports "payment succeeded but order failed." The copilot must identify affected users, trace the root cause across payment, order, and inventory events, and recommend fixes based on similar past incidents. Answering this query requires multi-hop reasoning over heterogeneous evidence sources (*e.g.,* transaction records, order-service failure traces, and historical tickets and runbooks), by traversing and aggregating distributed evidence under strict latency constraints. The answer is useful only if it is both correct and delivered within the TTFT/SLO budget. □

To handle such complex queries, prior work has evolved along two directions, as summarized in Figure 1. On the *retrieval side*, GraphRAG paradigms [16, 37] and neuro-symbolic approaches (*e.g.,* HippoRAG [19]) capture long dependency chains by constructing dense subgraphs or leveraging Personalized PageRank. These methods improve reasoning depth and reduce hallucinations by incorporating richer structural context, but primarily optimize retrieval recall (*i.e.,* the fraction of relevant evidence retrieved) with limited awareness of downstream execution cost. Thus, they often retrieve large contexts (*e.g.,* dense neighborhoods near "supernodes"), increasing prefill latency and inference memory footprint.

Conversely, on the *serving side*, systems such as vLLM [32] and SGLang [66] improve inference throughput via techniques like PagedAttention and RadixAttention, enabling efficient continuous batching and prefix sharing. Recent systems (*e.g.,* RAGO [28] and RAGCache [29]) further optimize configuration and caching for RAG workloads. However, these approaches linearize retrieved evidence into a flat token sequence, ignoring graph topology and per-query reasoning difficulty. As a result, they incur near-uniform compute even when only a small subset of evidence suffices.

Although emerging efforts have attempted to bridge this gap, they remain constrained by unidirectional data flows. For instance, DepCache [60] leverages dependency graphs to prune attention computation during inference, and FusionRAG [51] optimizes context splicing for cache reuse. Yet, these methods operate reactively at the serving layer; they cannot proactively regulate the upstream retrieval budget based on real-time system contention, nor can they prioritize critical reasoning paths under strict latency constraints.

This disconnect leads to a brittle *"Phase-Silo" Design* that optimizes intermediate metrics misaligned with end-user experience:

- Retrieval blindness: Retrieval modules maximize recall without accounting for serving-time cost. Expanding dense neighborhoods can inflate context length, triggering retrieval floods that immediately violate TTFT SLOs.
- Serving agnosticism: Schedulers optimize *throughput* (tokens generated per second) but ignore *goodput*, *i.e.,* the sustainable request rate under latency SLOs [67]. This can prioritize short queries while starving multi-hop ones, causing timeouts or incomplete reasoning.

This problem has received sustained attention in the data management community [1, 46, 51, 60]. The core challenge is cross-layer coordination: *how much evidence to retrieve per query and when to schedule it, guided by data structure and system state.*

[1] https://anonymous.4open.science/r/DynaGraph-Releasee-0496.

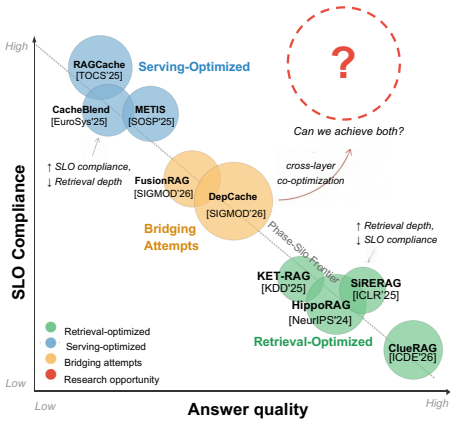


Figure 1: Quality–latency trade-off in RAG. Retrieval-optimized methods [16, 19, 22, 63] (green) boost answer quality but often miss SLOs; serving-optimized methods [28, 29, 44, 59] (blue) reduce latency but weaken retrieval depth. Bridging efforts [51, 60] (orange) narrow the gap but lack cross-layer feedback. Can we get both?

Our proposal. To bridge these silos, we propose DynaGraph, a holistic serving framework that turns linear RAG into a cooperative closed loop. Rather than optimizing retrieval recall and serving throughput in isolation, DynaGraph establishes a *bidirectional feedback channel* between the data and compute layers, using graph topology as a serving-time control signal to jointly coordinate evidence budgeting and scheduling under SLOs, where the topology reflects retrieval difficulty and predicts cost sensitivity, yielding a unified control prior for both decisions.

DynaGraph *redesigns retrieval to be compute-aware.* Instead of forwarding raw chunks, the retriever characterizes each retrieved subgraph's topological complexity (*e.g.,* high-degree "supernodes" or dense cliques) and exposes these signals as *cost priors* for scheduling. We also use a lightweight cost model that maps per-query evidence budgets to predicted prefill latency, enabling the aggregation controller and scheduler to estimate TTFT impact before inference.

DynaGraph *introduces serving-driven backpressure.* The inference engine monitors SLO slack and hardware contention and, as saturation approaches, feeds back to the aggregation layer to tighten evidence budgets for new queries. This turns a rigid recall–latency conflict into an adaptive trade-off: prune context for latency-sensitive workloads while reserving compute for hard multi-hop queries that require deeper evidence chains.

In summary, this paper makes the following contributions:

- *Problem formulation with a new metric (Section 2):* We introduce *QA-Goodput*, a quality-weighted serving metric that couples answer quality with hard TTFT deadlines. Based on this metric, we formulate a joint optimization problem that *simultaneously* selects per-query evidence budgets and an SLO-aware serving order to maximize QA-Goodput, and we show that this problem is NP-hard.
- *Preliminary study (Section 3):* Through a pilot online serving study, we show that phase-silo GraphRAG breaks down under strict and heterogeneous SLOs: fixed retrieval/aggregation budgets cannot handle skewed per-query cost and difficulty; scheduling alone mainly redistributes latency without reducing load; and congestion leads to head-of-line blocking that degrades end-to-end availability. These findings motivate a *joint*, feedback-driven design that couples cost control with SLO-aware scheduling.
- *Closed-loop pipeline (Section 4):* We design DynaGraph, a bidirectional feedback pipeline with three phases: (1) In Phase 1 (*Sensor*), it extracts lightweight topological signals during retrieval which can be used as serving-time control signals for downstream budgeting and scheduling; (2) In Phase 2 (*Controller*), it performs backpressure-driven adaptive aggregation, leveraging a lightweight cost model to estimate request cost and determine per-query budgets under real-time congestion guided by topology-derived signals; (3) In Phase 3 (*Actuator*), it applies an SLO-aware scheduler that ranks queries by urgency and estimated prefill cost with topology-informed difficulty, while feeding congestion signals back to the controller.
- *Extensive evaluation (Section 5):* We implement DynaGraph as a middleware that requires no LLM or retriever fine-tuning at serving time and that is portable across models and serving backends, requiring only lightweight calibration atop modern LLM serving engines, and evaluate it across benchmarks, models, workloads, and SLO regimes. Across three benchmarks and five LLM architectures, DynaGraph achieves 2.4×–3.3× higher QA-Goodput and reduces median TTFT by 56%–64% over the strongest baseline, while maintaining sub-linear end-to-end latency under 7× corpus growth.

We review related work in Section 6 and conclude in Section 7.

2 PROBLEM FORMULATION

In this section, we first introduce the standard graph-enhanced RAG pipeline, then present a new metric, *QA-Goodput*, that couples answer quality with hard TTFT deadlines, and finally formalize the problem studied in this paper.

2.1 Graph-Enhanced RAG Pipeline

Knowledge corpus. The knowledge corpus can be modeled as a heterogeneous graph $\mathcal{G} = (\mathcal{V}, \mathcal{E}, \mathcal{T}_v, \mathcal{T}_e)$, where $\mathcal{V}$ is the set of nodes representing granular knowledge units (*e.g.,* documents, chunks, and entities), $\mathcal{T}_v$ specifies node types (*e.g., entity, document, chunk*), $\mathcal{E}$ is the set of edges capturing pairwise relationships between nodes, and $\mathcal{T}_e$ defines edge types (*e.g.,* hyperlink, co-occurrence, semantic similarity) [16].

Pipeline. Given a graph-modeled corpus, a graph-enhanced RAG pipeline answers a multi-hop query q in three phases:

Phase 1: Retrieval (R). The retriever constructs a query-specific subgraph $\mathcal{G}_q \subseteq \mathcal{G}$ by combining embedding similarity with graph traversal (*e.g.,* community detection [16]). $\mathcal{G}_q$ captures multi-hop evidence paths and exposes topological signals $\mathcal{F}_q$ (*e.g.,* connectivity density and bridge nodes) reflecting reasoning complexity, which prior work typically discards [16, 19, 37].

Phase 2: Aggregation (A). Given retrieved $\mathcal{G}_q$, the aggregator selects K evidence chunks X_{ctx} to form the prompt $P = [X_{\text{sys}}, X_{\text{ctx}}, q]$, where X_{sys} provides task-specific instructions. Most systems fix K, inducing a rigid trade-off between context richness and cost. Let L_q denote the prompt length, which determines prefill cost and TTFT. Since K dominates, we approximate $L_q \approx |X_{\text{sys}}| + K \cdot \bar{\ell} + |q|$, where $\bar{\ell}$ is the average chunk length in X_{ctx}.

Phase 3: Generation (G) and Serving. Given prompt P, the LLM generates answer Y via prefill (KV cache construction, which domi-

nates TTFT) followed by decoding. Prefill cost grows super-linearly with L_q, so larger K increases latency. In practice, queries arrive stochastically at rate $\lambda(t)$ with heterogeneous complexity (*e.g.,* K=5 for a single-hop lookup vs. K=20 for a multi-hop query), inducing large cost variance. Under concurrency, requests compete for GPU resources, requiring batching, KV caching, and scheduling to meet latency targets. We focus on locally served open-source LLMs (*e.g.,* Qwen, Mistral) to enable fine-grained control.

Example 2: Continuing Example 1, the query is handled by a standard three-phase pipeline:
∘ *Phase 1 (Retrieval):* Starting from payment and order entities, the retriever extracts a subgraph $\mathcal{G}_{q'}$ linking transactions, service dependencies, failure traces, and incident signatures. The presence of a high-degree node (*e.g.,* "Error 503") indicates a dense fault region requiring more evidence to resolve.
∘ *Phase 2 (Aggregation):* The aggregator selects K chunks (*e.g.,* = 5) from $\mathcal{G}_{q'}$ to form $P_{q'} = [X_{\text{sys}}, X_{\text{ctx}}, q']$, where small K may miss cross-service dependencies; and large K increases cost and latency.
∘ *Phase 3 (Generation and Serving):* The engine executes $P_{q'}$ via prefill and decoding. Under FCFS scheduling, this query may be delayed by low-urgency requests, risking TTFT/SLO violations. □

Limitations. Modern serving engines (*e.g.,* vLLM [32] and SGLang [66]) improve throughput via KV-cache management, but remain inadequate for GraphRAG workloads with highly variable query difficulty and context size (see Section 3).

Cost-Blind Scheduling. Schedulers (*e.g.,* FCFS) treat requests as homogeneous, ignoring cost variance from topology, prompt length, and reasoning depth. For instance, a multi-hop query over a dense subgraph (*e.g.,* a supernode) may require far more context and prefill time than a single-hop lookup, yet both receive equal priority. This leads to *head-of-line (HOL) blocking* [30], where expensive queries delay latency-sensitive ones (or vice versa).

Static Context Budget. Most pipelines fix the context budget K, selecting the same number of chunks regardless of query difficulty or system load. Large K inflates prefill latency under contention, while small K wastes capacity and harms quality under light load. No existing system adapts K to real-time conditions.

No Cross-Layer Feedback. Retrieval, aggregation, and serving operate as an open loop: the serving layer cannot feed congestion signals upstream to adjust K, and upstream stages cannot provide cost signals to guide scheduling. As a result, the system cannot adapt to load fluctuations or heterogeneous query complexity.

Failure modes. Given the above limitations, existing approaches exhibit two common failure modes.

Fast but wrong (mode (a)). With insufficient context (*i.e.,* small K), LLM lacks key evidence and may hallucinate [3]. For reasoning-augmented models, missing grounding can also trigger longer exploration, increasing decode cost without improving accuracy [34].

Accurate but late (mode (b)). With excessive context (*i.e.,* large K), prefill dominates latency: answers are often correct but arrive late.

Both modes render responses ineffective: (a) arises for complex queries with small K, while (b) occurs for simple ones with large K.

Example 3: Continuing Example 2 with an SLO threshold τ=2 s, consider query under two static configurations:
∘ With K=5, the retriever retains only payment-callback logs and misses the upstream inventory-lock root cause; the LLM thus hallucinates a "gateway timeout" (TTFT ~180 ms; failure mode (a)).
∘ With K=20, all relevant evidence is included and the root cause is correctly identified, but the ~4,000-token prompt raises TTFT to ~2,800 ms, exceeding the SLO (failure mode (b)). □

2.2 Metric and Problem Formulation

In this subsection, we introduce a new evaluation metric for graph-enhanced RAG, upon which we formalize our problem.

Metric. We first review the widely used Goodput metric and its limitations, and then introduce a new metric for graph-enhanced RAG that jointly accounts for answer accuracy and a hard deadline.

Goodput [67]. It quantifies the maximum sustainable request arrival rate subject to a latency Service Level Objective (SLO), specified as a per-request deadline τ_q on TTFT (time-to-first-token):

$$\text{Goodput} = \max\ \lambda \quad \text{s.t.} \quad \Pr\big(\text{TTFT} \le \tau_q\big) \ge \beta \tag{1}$$

where λ is the request arrival rate and $\beta \in (0, 1]$ is the target SLO-attainment probability.

As shown in Eq. (1), Goodput accounts only for timeliness; it treats a correct on-time answer and an on-time hallucination equally, leaving failure mode (a) unaddressed.

QA-Goodput. To capture the failure modes in Section 2.1, we introduce *QA-Goodput* $\mathcal{J}_\lambda$, which (a) assigns zero credit to any response that misses its deadline, regardless of correctness, and (b) penalizes low-quality answers. Formally, for a query set $\mathcal{S} = \{q_i\}_{i=1}^N$ at arrival rate λ, where each query q_i has a TTFT deadline τ_{q_i}, we define:

$$\mathcal{J}_\lambda = \frac{1}{N}\sum_{i=1}^{N} \underbrace{\text{Score}(q_i)}_{\text{quality}} \cdot \underbrace{\mathbb{I}(\text{TTFT}_i \le \tau_{q_i})}_{\text{timeliness}} \tag{2}$$

where $\text{Score}(q_i) \in [0, 1]$ measures answer quality (*e.g.,* token-level F1 [43]), and $\mathbb{I}(\cdot)$ is an indicator function. Intuitively, a higher $\mathcal{J}_\lambda$ indicates that more queries in $\mathcal{S}$ are answered accurately and within their deadlines, thereby mitigating both failure modes.

When late answers retain partial value, one may instead use a soft time utility with bounded linear decay, *e.g.,* [25]:

$$\mathcal{J}_\lambda^{\text{soft}} = \frac{1}{N}\sum_{i=1}^{N} \text{Score}(q_i) \cdot w(\text{TTFT}_i, \tau_{q_i}).$$

Here, $w(\text{TTFT}i, \tau q_i)$ is a function that assigns full credit (*i.e.,* 1) to responses meeting the deadline τ_{q_i}, linearly decaying credit to those arriving between τ_{q_i} and $2\tau_{q_i}$, and zero credit thereafter.

Note. (1) Goodput considers only timeliness, counting any on-time response as successful regardless of correctness. In contrast, QA-Goodput couples answer quality with timeliness, thereby complementing Goodput. (2) Hard-deadline QA-Goodput applies when late responses have no utility, whereas soft time utility applies when late responses retain partial value. The two metrics are complementary, and their suitability depends on the deployment setting.

Problem. We formalize the problem studied in this paper as follows.
∘ *Input:* a document corpus $\mathcal{D}$ (from which a heterogeneous graph $\mathcal{G}$ is constructed), and a query batch $\mathcal{S} = \{q_i\}_{i=1}^n$ arriving at rate

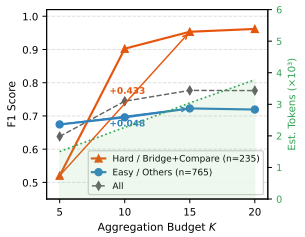


Figure 2: Static configuration under varying K. Green shading: prefill tokens (right axis).

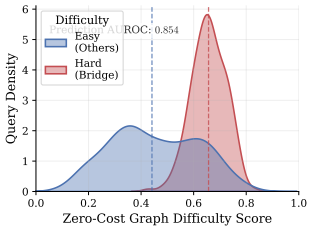


Figure 3: Zero-cost difficulty prediction. Difficulty score distribution for Easy (n=765) and Hard (n=235); AUROC = 0.854.

λ, where each query q_i has a TTFT deadline τ_{q_i} and an admissible retrieval/aggregation budget $K_i \in [K_{\min}, K_{\max}]$.

∘ *Output:* a serving order π of queries in $\mathcal{S}$ and per-query budgets $\mathcal{K} = \{K_1, \ldots, K_n\}$.

∘ *Objective:* maximize QA-Goodput under query budget constraints:

$$\max_{\mathbf{K},\pi} \sum_{i=1}^{n} Q(K_i, q_i) \cdot \mathbb{I}(\mathsf{TTFT}_i \le \tau_{q_i}) \quad \text{s.t.} \quad K_i \in [K_{\min}, K_{\max}],\ \forall i \quad (3)$$

where $Q(K_i, q_i)$ denotes the answer score (cf. Eq. (2)) when serving q_i with K_i evidence chunks.

Note. (1) The problem requires jointly optimizing two coupled decisions: the serving order π (scheduling) and the per-query budgets $\mathcal{K}$ (budget allocation). (2) The interval $[K_{\min}, K_{\max}]$ defines feasible system budgets and can be set from prior workload knowledge. (3) Since scheduling and budget allocation jointly influence both quality and timeliness, the problem is more challenging than standard deadline scheduling with fixed processing times (see below).

Hardness. The restricted decision version of the studied problem in Eq. (3) is to decide, given a query set $\mathcal{S}$, arrival rate λ, per-query deadlines τ_{q_i} for each $q_i \in \mathcal{S}$, a fixed budget vector $\mathbf{K}$, and a threshold η, whether there exists a serving order π on a non-preemptive single machine such that $\sum_{i=1}^{n} Q(K_i, q_i) \cdot \mathbb{I}(\mathsf{TTFT}_i \le \tau_{q_i}) \ge \eta$. We show that even this restricted version is intractable.

Theorem 1: *The above restricted decision problem is NP-hard.* □

Proof. We establish NP-hardness by reducing from Partition [31]. Proof details are deferred to Appendix B.1 in [4].

3 MOTIVATION AND ANALYSIS

In this section, we present the test setup (Section 3.1), empirical evidence of their limitations (Section 3.2), and key observations motivating our approach (Section 3.3).

3.1 Setup

Dataset. We use 2WikiMultihopQA [21], a two-hop QA benchmark with four reasoning types: *compositional* (n=413), *comparison* (n=244), *bridge_comparison* (n=235, requiring cross-document bridging and comparison), and *inference* (n=108). We treat *bridge_comparison* as "Hard" and the remaining types as "Easy", as they are less sensitive to the context budget K. This dataset's annotated reasoning types enable controlled stratification by difficulty.

Workload. Requests arrive via a Poisson process at rate λ [29, 44], with queries drawn from a Zipfian popularity distribution (α=1.0) to reflect the skewed entity distribution observed in knowledge-graph workloads [7, 8]. This amplifies prompt-length variance and HOL blocking, leading to a harder scheduling problem than uniform.

Configurations. We evaluate four settings: *Static (C1)* uses fixed K=20 with FCFS (phase-silo baseline); *Schedule-Only (C2)* replaces FCFS with SLO-aware scheduling, isolating scheduling; *Adaptive-K (C3)* applies difficulty-aware K with FCFS (expert-provided difficulty), isolating budget adaptation; *Joint (C4)* combines adaptive K, SLO-aware scheduling, and load-dependent backpressure under expert-defined policies. These isolate *budget adaptation* (C1→C3) and *scheduling* (C1→C2), with C4 evaluating their joint effect.

Infrastructure. All experiments run on a single NVIDIA RTX 4090 (24 GB). We use Qwen3-4B with Zipfian arrivals (λ=30 req/s; 1000 queries × 3000 requests; concurrency 64).

Quality Metric. Answer quality is measured by token-level F1 [43], corresponding to Score(q_i) in the QA-Goodput definition (Eq. 2).

3.2 Observations

Under the test setup in Section 3.1, we observe the following.

(1) The context size K matters. We varied the aggregation budget K (5–20) for the static configuration (C1) across four reasoning types (Figure 2). (a) The quality–cost trade-off is strongly difficulty-dependent: hard queries gain substantially from larger K, while easy queries gain little. For instance, increasing K from 5 to 15 raises hard-query F_1 from 0.52 to 0.95, *e.g.*, 9× the gain for easy queries, since small K often misses evidence for multi-step reasoning. (b) Larger K can also hurt: increasing K from 15 to 20 reduces F_1 for easy queries, suggesting that oversized contexts introduce noise. Thus, fixed-K aggregation wastes budget on easy queries and underserves hard ones, making difficulty-aware K essential.

(2) Adaptive K alone improves efficiency but may violate strict SLOs. We compared adaptive-K (C3) against Static (C1) to test whether difficulty-based K improves efficiency without harming accuracy or SLOs. In Table 1, with expert difficulty labels, C3 reduces avg K from 20.0 to 14.7 (−26%) with negligible F1 loss (−0.008). Under a moderate SLO (τ=500 ms), C3 reaches 91.9% compliance and $\mathcal{J}_\lambda$@500 = 74.0% (+26.8pp over C1); under a strict SLO (τ=200 ms), it still misses 54% of deadlines ($\mathcal{J}_\lambda$@200 = 38.3%). This is because FCFS ignores urgency: adaptive K reduces per-query cost but cannot prevent HOL blocking. Moreover, C3 assumes serving-time difficulty labels, which is unrealistic; prior methods [47] rely on similarity signals and lack load-aware feedback.

(3) Scheduling alone is insufficient. Since FCFS still violates strict SLOs even with adaptive K (Observation (2)), we evaluated whether scheduling alone can close the gap. Configuration C2 applies an SLO-aware priority policy while keeping K=20 fixed. In Table 1, prioritization reduces median latency (P50: 426→361 ms) but worsens the tail (P99: 2,796→6,484 ms, 2.3×), because deprioritized queries still pay the full prefill cost and can wait unbounded under contention. As a result, QA-Goodput improves only slightly: $\mathcal{J}_\lambda$@200 increases by +2.5pp (16.3→18.8%), far below the +22.0pp gain from C3. Thus, scheduling reshapes latency but cannot reduce prefill cost.

(4) Joint optimization is promising. Combining adaptive K and SLO-aware scheduling (C4) yields the best results: $\mathcal{J}_\lambda$@200 = 61.4% (3.8× over C1), $\mathcal{J}_\lambda$@500 = 78.4%, and lowest P50 (136 ms, −68%), with small F1 loss (−0.017). Improvements over C3 (+28.5pp SLO@200;

Table 2: Summary of key notations.

Symbol	Description
$\mathcal{S}, \lambda$	Query batch and request arrival rate
$\mathcal{J}_\lambda$	QA-Goodput (Eq. 2)
$\mathbf{K}, \pi$	Budget vector and scheduling permutation (decision variables)
$\mathcal{G}$; q, $\mathcal{G}_q$	Knowledge graph; query and induced subgraph
$\mathcal{F}_q$	Topological feature vector of $\mathcal{G}_q$
$S_{\text{diff}}(q)$	Difficulty score $\in [0, 1]$ (Eq. 5)
K_q; $K_{\min}, K_{\max}, K_{\text{eff}}$	Query budget; lower, upper, and load-adjusted bounds
$u(t)$, θ_{bp}	Queue utilization and backpressure threshold
$L_i(K_i)$, μ_{p}	Prompt length and prefill throughput (Eq. 4)
τ	TTFT deadline (SLO)

Table 1: Motivation: configuration comparison. (Qwen3-4B; 2WikiMultiHopQA; Zipf λ=30 req/s; 1000q×3000 requests.)

Config	Quality		Latency (↓)		τ=200 ms (↑)		τ=500 ms (↑)	
	F1	avg K	P50	P99	SLO	$\mathcal{J}_\lambda$	SLO	$\mathcal{J}_\lambda$
Static (C1)	0.758	20.0	426 ms	2796 ms	18.7%	16.3%	57.2%	47.2%
Sched-Only (C2)	**0.762**	20.0	361 ms	6484 ms	21.6%	18.8%	66.0%	54.9%
Adaptive-K (C3)	0.750	14.7	218 ms	**2224 ms**	45.9%	38.3%	91.9%	74.0%
Joint (C4)	0.741	**12.5**	**136 ms**	2288 ms	**74.4%**	**61.4%**	**97.8%**	**78.4%**

+4.4pp $\mathcal{J}_\lambda$@500) show that scheduling amplifies adaptive K. However, C4 depends on hand-tuned labels/policies, limiting practicality.

(5) Query difficulty can be captured by graph topology. To make C4 practical, we need to estimate per-query difficulty at serving time without expert labels or extra inference. Prior work shows graph properties correlate with QA difficulty [26, 58, 62]. We therefore use retrieval-time subgraph statistics—zero-cost byproducts of graph traversal—as proxies. As shown in Figure 3, a linear combination of three topological features (Section 4.2) achieves AUROC = 0.854 in separating Hard from Easy queries, without LLM inference or task-specific training. This is because multi-hop queries induce structurally distinct subgraphs, largely independent of surface phrasing.

3.3 Summary and Insights

Summary. The key findings from Section 3.2 are as follows.

(1) *Adaptive aggregation is the primary lever for the quality–cost trade-off.* Observation (1) shows that a fixed context budget wastes tokens on easy queries while under-serving hard ones. Difficulty-aware budgeting reduces avg K from 20 to 14.7 with only −0.008 F1 loss, yielding 2.4× higher $\mathcal{J}_\lambda$@200ms than Static, and significantly improving latency (P50: 426→218 ms, −49%).

(2) *Strict SLOs require scheduling coupled with backpressure.* Observations (2–3) show that scheduling alone improves median latency but worsens the tail (P99: 6,484 ms). In contrast, combining SLO-aware scheduling with adaptive K and backpressure feedback yields an observable gain in SLO@200ms over Adaptive-K alone (45.9%→74.4%), as backpressure compresses budgets under congestion and prevents queue buildup.

(3) *Graph topology provides accurate, zero-cost difficulty signals.* Observation (5) shows that retrieval-time subgraph features separate Hard vs Easy with AUROC = 0.854 (Obs. 5), enabling difficulty scoring and priority assignment without extra model inference.

Insights. (a) Graph topology effectively captures query difficulty, enabling lightweight, retrieval-time signals to guide budgeting and replace impractical expert tuning; we thus need to study which features are most predictive and stable. (b) Beyond controlled settings, real deployments involve large, heterogeneous workloads with varying deadlines, requiring joint, online adaptation of budgets and scheduling; achieving high QA-Goodput thus necessitates a closed-loop coupling of aggregation and serving.

4 METHODOLOGY

In this section, we overview our system (Section 4.1), detail its main components (Sections 4.2–4.4), and present a closed-loop algorithm with guarantees (Section 4.5).

4.1 System Overview

Overview. We present DynaGraph, a holistic serving framework for maximizing QA-Goodput (Section 2.2) under stochastic arrival loads. As shown in Figure 4, unlike feed-forward pipelines, it establishes a *bidirectional feedback loop* between data complexity (Graph) and system capacity (Serving), and decomposes the joint problem (Eq. 3) into three coordinated phases.

Given a query q, the system first retrieves a query-specific subgraph and extracts lightweight topological features that capture its structural complexity (Phase 1). These signals are then used to estimate query difficulty and determine a budgeted context and prompt under current system load (Phase 2). Finally, the resulting requests are scheduled based on their urgency, estimated cost, and topology-informed difficulty, producing the execution order and inference results (Phase 3). This forms a closed loop in which topology-derived signals guide both budgeting and scheduling decisions.

The key ideas of the three phases are as follows.

Phase 1: Retrieval & Signal Extraction (Sensor, Section *4.2).* Given query q and corpus $\mathcal{D}$, this phase produces: (a) a heterogeneous graph $\mathcal{G}$ (offline), (b) a query-induced subgraph $\mathcal{G}_q \subseteq \mathcal{G}$, and (c) a topological feature vector $\mathcal{F}_q$ as a proxy for query complexity (Obs. (5)). These lightweight signals guide budgeting in Phase 2 and scheduling in Phase 3 before LLM inference.

Phase 2: Adaptive Aggregation (Controller, Section *4.3).* Given $\mathcal{G}_q$, $\mathcal{F}_q$, and system backpressure S_{sys} (from the serving engine in Phase 3), this phase outputs budget K_q and prompt P_q with length L_q. It balances difficulty and load (Obs. (1)), allocating more context to hard queries and throttling easy ones under congestion, selecting K_i to maximize expected quality $Q(K_i, q_i)$.

Phase 3: SLO-Aware Scheduling (Actuator, Section *4.4).* Given $\{P_q\}$, SLOs $\{\tau_{q_i}\}$, and estimates ($\mathcal{F}_q$, S_{diff}, L_q), this phase outputs serving order π and results Y. It replaces FCFS with an SLO-aware priority queue that ranks queries by urgency and cost, mitigating HOL blocking (Obs. (2–3)) and maximizing QA-Goodput under $\mathbf{K}$.

Note. (1) Phase 3 feeds congestion signals S_{sys} (*e.g.,* queue depth) back to Phase 2, closing the feedback loop for jointly optimizing the evidence budgets $\mathbf{K}$ and execution order π under load. The backward feedback acts only on K_q (and thus the prompt length L_q), without changing Phase 1 retrieval. (2) Sensor (priors), Controller (allocation) and Actuator (execution) form a feedback-control system [20] that adapts to workload dynamics without online retuning.

4.2 Retrieval & Signal Extraction (The Sensor)

Overview. Given query q and corpus $\mathcal{D}$, this phase builds (if needed) an offline heterogeneous graph $\mathcal{G}$ with Document/Chunk/Entity nodes, induces a query subgraph $\mathcal{G}_q$ via two-hop

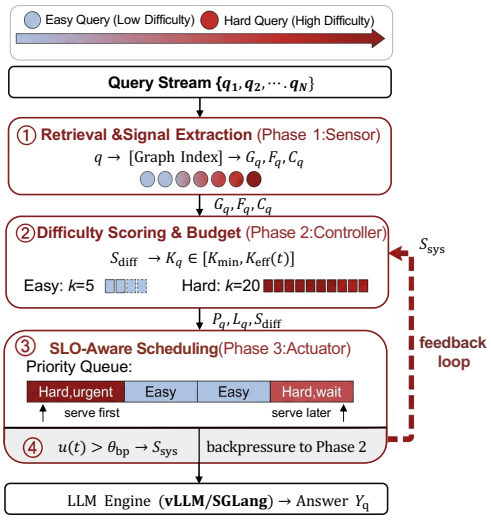


Figure 4: DynaGraph Architecture. Phase 1 (Sensor) extracts topological priors from the graph index; Phase 2 (Controller) budgets context K_q based on difficulty and backpressure; Phase 3 (Actuator) schedules by urgency and cost, feeding congestion signals back to Phase 2.

structure-semantic retrieval (semantic anchoring + graph expansion), jointly re-ranks candidates into C_q, and extracts a lightweight topological feature vector $\mathcal{F}_q$ as structural priors for downstream budgeting and scheduling.

Heterogeneous graph construction. Given corpus $\mathcal{D}$, we construct (if not already available) an offline heterogeneous graph index $\mathcal{G} = (\mathcal{V}, \mathcal{E}, \mathcal{T}_v, \mathcal{T}_e)$ (defined in Section 2.1) in three stages. First, each document $d \in \mathcal{D}$ is segmented into semantic chunks that preserve sentence boundaries [24]. Second, an offline LLM extracts entities and relational triples from each chunk via schema-constrained pipelines [12, 41, 61], producing canonicalized entity nodes grounded to a reference ontology (*e.g.,* Wikidata [50]). Third, we construct the heterogeneous graph by linking documents, chunks, and entities, forming inclusion edges (Document→Chunk→Entity) and relational edges (Entity↔Entity). This structure can expose *supernodes* (high-degree entities causing prompt variance; Obs. (2)) and *bridges* (cross-region connectors requiring larger K; Obs. (1)).

Structure-Semantic Collaborative Retrieval. Given q and $\mathcal{G}$, it induces a candidate subgraph $\mathcal{G}_q \subseteq \mathcal{G}$ and a ranked candidate corpus C_q via a two-hop coarse-to-fine strategy (seed-expand) [19, 22, 46]: (1) *Hop-1 (semantic anchoring)* retrieves the top-K_1 chunks by bi-encoder cosine similarity to form C_q^{chunk}, expands the parent documents of these chunks into their constituent chunks to form the document-expanded chunk set C_q^{doc}, and traverses inclusion edges to gather mentioned entities as the seed set; (2) *Hop-2 (structure-guided expansion)* traverses up to two relational hops from the seeds to surface bridging evidence, then maps newly discovered entities back to their source chunks via inclusion edges to form C_q^{graph}, with $\mathcal{G}_q$ defined as the subgraph induced by Hop-1 anchoring and Hop-2 expansion, *i.e.,* the anchor chunks, their document/entity nodes, expanded entities, and all traversed inclusion/relational edges; and (3) *unified scoring* jointly re-scores the candidate set $C_q = C_q^{\text{chunk}} \cup C_q^{\text{doc}} \cup C_q^{\text{graph}}$ using a cross-encoder reranker to produce $s_{\text{rerank}}(d, q)$ and a single ranked list, while retaining bi-encoder embeddings $\mathbf{e}_d$ for Phase 2 (Section 4.3.3).

Each element of C_q is an LLM-consumable chunk-level evidence unit; documents/entities are used to induce retrieval context, graph expansion, and topology signals, not as downstream evidence items.

Topological signal extraction. We map $\mathcal{G}_q$ to features $\mathcal{F}_q \in \mathbb{R}^d$ as zero-cost structural priors for downstream control (Phases 2–3): they are inexpensive to compute and informative for initialization, though not used as standalone predictors. Let $\mathcal{E}(c)$ denote entities in chunk c, and $C_q^{\mathcal{G}}$ the chunk set. We introduce three key signals in $\mathcal{F}_q$ that will be used in later phases.

- *Bridge ratio* r_{bridge}: fraction of chunks sharing entities with ≥ 2 others; indicates cross-region connectivity and multi-hop complexity (Obs. 1), and provides the strongest single-feature signal.
- *Entity overlap density* ρ_{overlap}: fraction of chunk pairs sharing entities, capturing local connectivity and complementing r_{bridge}.
- *Entity count* n_{entity}: number of unique entities in $\mathcal{G}_q$, reflecting evidence breadth and reasoning depth.

Example 4: For the e-commerce support query q' ("payment succeeded but order failed …"), Phase 1 induces a subgraph $\mathcal{G}_{q'}$ that links evidence around *order*, *payment*, *inventory*, and *ticket*. It then extracts the corresponding feature vector $\mathcal{F}_{q'}$ (*e.g.,* flagging the "Error 503" entity as a supernode). The Controller (Phase 2) and Actuator (Phase 3) consume these signals to drive difficulty-aware budgeting and scheduling before the expensive prefill stage. □

Note. (1) Extracting $\mathcal{F}_q$ involves only simple combinatorial operations over a small subgraph (typically $<$ 500 nodes) and adds negligible overhead (typically $<$ 5 ms in our implementation). This provides Phase 2 with an early signal that helps disentangle quality-driven budget needs from system-driven cost control. (2) DynaGraph is agnostic to graph construction, provided that the index exposes document/chunk/entity structure from which Phase 1 can derive query-induced topology signals. For enterprise domains where general-purpose ontologies may not apply, domain-specific ontology extraction and validation support ontology-aware KG construction from unstructured corpora [12, 35]. Industrial-scale continuous KG platforms [23] can then maintain the resulting index. Streaming updates use micro-batch or incremental refresh of affected documents/entities while serving uses the latest consistent graph snapshot. Co-design of ontology construction, incremental graph maintenance and SLO-aware serving is left to future work.

4.3 Adaptive Aggregation (The Controller)

This subsection presents Phase 2, which acts as the *controller* between retrieval (Phase 1) and serving (Phase 3). It maps Phase 1 outputs $(\mathcal{G}_q, \mathcal{F}_q, C_q)$ to a query-specific budget K_q and an LLM-ready prompt under real-time backpressure.

We first establish a cost model linking K_q to prefill cost (Section 4.3.1), which underpins the budget control strategy. We then introduce a topology-based difficulty estimator and a backpressure-aware budget allocation method (Section 4.3.2). Finally, we describe a context assembler that selects and organizes K_q evidence units under the allocated budget (Section 4.3.3).

4.3.1 *Cost Model.*

To quantify how K_q affects serving cost and motivate adaptive budgeting, we model LLM inference cost. In autoregressive inference, each request incurs two sequential costs with distinct profiles [2, 67]. For query q_i with budget K_i, the end-to-end GPU occupancy is:

$$c_i(K_i, q_i) = \underbrace{L_i(K_i) / \mu_p}_{\text{prefill (compute-bound)}} + \underbrace{\hat{D}_i(q_i) / \mu_d}_{\text{decode (memory-bound)}}, \quad (4)$$

where $L_i(K_i) = |X_{sys}| + |q_i| + K_i \cdot \bar{\ell}$ is the prompt length, μ_p and μ_d are prefill and decode throughput (tokens/ms), and $\hat{D}_i(q_i)$ is the expected output length. The prefill term is *controllable*: K_i determines $L_i(K_i)$ and thus TTFT_i [28, 44]. The decode term is not directly controllable and is observed only during generation.

Note. (1) This asymmetry motivates a two-lever design: Phase 2 controls prefill via K_i, while Phase 3 monitors queue utilization to handle combined prefill and decode effects. (2) The subgraph $\mathcal{G}_{q_i}$ from Phase 1 is informative for setting K_i: larger neighborhoods, higher branching, and fragmented structure require more evidence, motivating the topology-based difficulty estimator (Section 4.3.2).

4.3.2 Budget Allocation under Backpressure.

We introduce (a) a *cost-aware difficulty estimator* using Phase 1 topology signals, (b) a *dynamic budget allocation* that maps difficulty and real-time utilization (Phase 3) to the final budget K_q, and (c) a *lightweight optional retrieval-ambiguity guardrail* for topology-induced false "easy" cases before final budget allocation.

Cost-aware difficulty estimator. To distinguish complex queries from simple ones, we define a scoring function $S_{\text{diff}} : \mathcal{Q} \to [0, 1]$ based on topology features extracted in Phase 1 (*i.e.,* bridge ratio r_{bridge}, overlap density ρ_{overlap}, and entity count n_{entity}). Let $\tilde{\mathbf{f}}_q$ denote the min–max normalized feature vector; we compute

$$S_{\text{diff}}(q) = \mathbf{w}^\top \tilde{\mathbf{f}}_q, \quad (5)$$

where $\mathbf{w} \in \mathbb{R}^3$ is selected via grid search on query-grouped calibration data (see Appendix E.8 in [4] for the calibration protocol and sensitivity analysis). $S_{\text{diff}}(q) \approx 1$ indicates a hard query (*e.g.,* substantial cross-cluster bridging, moderate overlap, and many entities), whereas $S_{\text{diff}}(q) \approx 0$ indicates a simple lookup (*e.g.,* minimal bridging and low overlap).

Note. (1) S_{diff} is a *coarse and continuous* classifier (hard vs. easy), not an oracle regressor for K^*. To guard against false "easy" cases under topology-only scoring, we later use retrieval-score dispersion as a lightweight check before final budget allocation (see below). (2) Features that separate difficulty (*e.g.,* r_{bridge}, ρ_{overlap}) do not localize gold evidence (which requires supervised selectors [15, 64], learned complexity models [26], or LLM profilers [44]); coarse ranking suffices as Phase 3 amplifies it into QA-Goodput gains. (3) S_{diff} proxies decode cost $\hat{D}_i(q_i)$ (Eq. 4), since more bridging typically induces longer reasoning chains [34]. (4) The estimator is *zero-cost and training-free* at serving time, reusing Phase 1 outputs with $\mathbf{w}$ calibrated via grid search on query-grouped calibration data.

Dynamic budget allocation. Given $S_{\text{diff}}(q)$, the controller maps difficulty to budget K_q via a two-regime control law: (R1) maximize quality under low load, and (R2) degrade gracefully under congestion while preserving difficulty ordering.

We quantify load by queue utilization $u(t) \in [0, 1]$, *i.e.,* the ratio of pending prefill workload to the scheduling-window capacity:

$$u(t) = \Big(\sum_{i \in Q(t)} p_i\Big)/\delta, \qquad p_i = L_i(K_i)/\mu_p, \quad (6)$$

where $Q(t)$ is the set of pending/active prefill requests and δ is the scheduling window.

Let $\theta_{\text{bp}} \in [0, 1]$ be a backpressure threshold separating two regimes: full budgets when $u(t) \le \theta_{\text{bp}}$, and budget compression otherwise. This couples K_q with real-time load, reducing aggregate prefill cost while preserving relative prioritization.

Quality-first regime. If $u(t) \le \theta_{\text{bp}}$, K_q depends only on difficulty:

$$K_q = K_{\min} + \lfloor (K_{\max} - K_{\min}) \cdot S_{\text{diff}}(q) \rceil. \quad (7)$$

We assign larger budgets to harder queries and smaller ones to easier queries, using a linear map to keep K unimodal under Zipf workloads and avoid bimodality [20]. We denote the quality-first budget in Eq. (7) as $K_{\text{diff}}(q)$, *i.e.,* the budget determined by query difficulty before backpressure is applied. In context assembly (Section 4.3.3), $K_{\text{diff}}(q)$ also defines the selection window $\mathcal{D}_q[1{:}K_{\text{diff}}]$ from which the final K_q evidence units are chosen under backpressure.

Congestion regime. If $u(t) > \theta_{\text{bp}}$, apply a load-dependent cap:

$$K_{\text{eff}}(t) = \begin{cases} K_{\max}, & u(t) \le \theta_{\text{bp}}, \\ K_{\min} + (K_{\max} - K_{\min}) \cdot \phi(t), & \text{otherwise}, \end{cases} \quad (8)$$

where $\phi(t) = \exp\Big(-\alpha_{\text{bp}} \cdot \dfrac{u(t) - \theta_{\text{bp}}}{1 - \theta_{\text{bp}}}\Big)$ and $\alpha_{\text{bp}} = \ln(K_{\max}/K_{\min})$.

Here the exponent $\phi(t)$ combines two normalized factors controlling budget contraction. The term $\frac{u(t)-\theta_{\text{bp}}}{1-\theta_{\text{bp}}} \in [0, 1]$ captures excess utilization (from congestion onset to saturation), while α_{bp} sets the compression range, ensuring $K_{\text{eff}}(t)$ decays from $K_{\max}$ toward $K_{\min}$. Together, they yield exponential decay that is mild near the threshold and increasingly aggressive toward saturation.

Eq. (8) can be viewed as an online knapsack reservation function [56, 68]; exponential reservation thresholds motivate our decay design (see Appendix in [4] for the theorem and Section 5.4 for comparisons with alternative decay shapes). By construction, $K_{\text{eff}}(t)$ decreases monotonically with $u(t)$, smoothly interpolating from $K_{\max}$ toward $K_{\min}$, while θ_{bp} defines a slack region that absorbs burstiness. Each query is allocated proportionally within $[K_{\min}, K_{\text{eff}}(t)]$:

$$K_q = K_{\min} + \lfloor (K_{\text{eff}}(t) - K_{\min}) \cdot S_{\text{diff}}(q) \rceil, \quad (9)$$

which preserves relative difficulty while compressing the budget range. When $u(t) \le \theta_{\text{bp}}$, this reduces to Eq. (7). Under congestion, Eq. (9) compresses the budget range while preserving the difficulty ordering: hard queries stay near the top of the compressed range rather than collapsing to $K_{\min}$, and Phase 3 further increases their scheduling urgency (Section 4.4).

Retrieval-ambiguity guardrail. Although $S_{\text{diff}}(q)$ provides a zero-cost topology prior, it can yield false "easy" cases: the query-induced graph may be sparse even when the retrieved textual evidence is ambiguous or conflicting. In such settings, topology-only scoring can over-compress the context budget and overlook quality risks not reflected in structural metrics. To reduce this risk, we optionally apply a lightweight guardrail before budget allocation. Let $\sigma_{\text{ret}}(q)$ denote the dispersion of the top retrieval scores in Phase 1; smaller values indicate flatter and less decisive evidence. If $S_{\text{diff}}(q)$ falls below a topology-easy threshold and $\sigma_{\text{ret}}(q)$ falls below a retrieval-

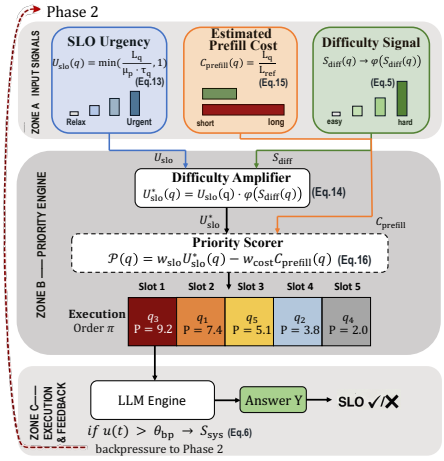


Figure 5: SLO-Aware Scheduling Logic. Priority $\mathcal{P}(q)$ combines deadline urgency U^*_{slo} (amplified by difficulty S_{diff} from Phase 2) and estimated prefill cost C_{prefill} to order the execution queue. A feedback monitor emits S_{sys} to Phase 2 when utilization exceeds θ_{bp}.

ambiguity threshold, the controller conservatively increases the budget assigned by the subsequent budget mapping, capped by the current budget ceiling. Thus, the guardrail provides a lightweight optional check for topology-induced false "easy" cases, while leaving confident topology-easy queries unchanged and requiring no additional model inference.

Note. (1) Adaptive K and priority scheduling are complementary: Phase 2 controls prefill workload to induce cost heterogeneity ($K_{\text{easy}} \ll K_{\text{hard}}$), which Phase 3 exploits when ordering requests under SLO constraints. (2) The system operates in three regimes: (a) *quality-first* ($u(t) < \theta_{\text{bp}}$), (b) *transition* ($u(t) \approx \theta_{\text{bp}}$), and (c) *goodput-first* ($u(t) \gg \theta_{\text{bp}}$), where exponential decay dominates.

4.3.3 Context Assembler.

Given budget K_q, the *assembler* selects K_q evidence units from C_q and forms an LLM-ready prompt for Phase 3.

Objective. Construct a size-K_q set that (a) maximizes answer-critical coverage and (b) minimizes redundancy, jointly covering diverse regions of $\mathcal{G}_q$. We optimize a relevance-weighted coverage objective under a cardinality constraint via diversity-aware greedy selection.

Unified scoring and selection. For candidates $C_q = C_q^{\text{chunk}} \cup C_q^{\text{doc}} \cup C_q^{\text{graph}}$ (Phase 1), we deduplicate and assign each chunk-level evidence unit $d \in C_q$ a score:

$$s(d, q) = s_{\text{rerank}}(d, q) + \delta_g \cdot \mathbb{I}(d \in C_q^{\text{graph}}). \tag{10}$$

Here δ_g denotes the graph-source bonus. This yields a ranked list $\mathcal{D}_q$. If uncongested ($K_q = K_{\text{diff}}$), we apply top-K_q. Under backpressure ($K_q < K_{\text{diff}}$), we select K_q representatives from $\mathcal{D}_q[1{:}K_{\text{diff}}]$ via a relevance-weighted k-center objective:

$$\mathcal{S}_q^* = \arg \min_{|S|=K_q} \max_{d \in \mathcal{D}_q[1:K_{\text{diff}}]} s(d, q) \cdot \big(1 - \max_{d' \in S} \cos(\mathbf{e}_d, \mathbf{e}_{d'})\big), \tag{11}$$

where $\mathbf{e}_d$ is the Phase 1 embedding. The cosine term penalizes redundancy, while $s(d, q)$ preserves relevance. We solve Eq. (11) via farthest-first traversal [17] in $O(|C_q| \cdot K_q)$ (microseconds on CPU).

Context construction. We organize $\mathcal{S}_q^*$ by provenance and local proximity in $\mathcal{G}_q$, placing bridge entities near neighbors to support multi-hop reasoning. The final prompt is

$$P_q = [\, X_{\text{sys}},\ \mathcal{S}_q,\ q \,], \quad L_q = |P_q|. \tag{12}$$

with length $L_q = |P_q|$, passed to Phase 3 as the realized prefill cost.

Note. The assembler separates *how much* from *which*: topological signals determine global difficulty ($S_{\text{diff}} \rightarrow K_q$), while embedding-space coverage governs selection ($\mathcal{D}_q \rightarrow \mathcal{S}_q$). This preserves answer quality under constrained budgets, creates cost heterogeneity across queries, and enables Phase 3 to exploit it via SLO-aware scheduling.

4.4 SLO-Aware Scheduling (The Actuator)

Given budgets K_q from Phase 2, Phase 3 determines serving order π under SLO constraints to maximize QA-Goodput, acting as the *actuator* over shared GPU resources.

In the sequel, we begin by revisiting why FCFS breaks down under bursty arrivals and outlining the resulting challenges. We then present the scheduler's view, describing the signals it tracks and how they are combined into a priority score to address these challenges. Finally, we present a non-preemptive admission policy for GPU execution and the feedback loop through which signals from Phase 3 inform subsequent budget decisions in Phase 2.

Motivation. Phase 2 produces queries $\{q\}$ with prompts $\{P_q\}$, each with prefill cost p_q and difficulty $S_{\text{diff}}(q)$. Under bursty arrivals, HOL blocking dominates: a few large-L_q queries can stall the queue and trigger cascading TTFT violations. We therefore replace FCFS with an SLO-aware priority scheduler balancing urgency and cost.

Challenges. Phase 3 faces three challenges: *(1) Cost heterogeneity*: adaptive budgets introduce substantial variance in prefill cost; *(2) Deadline awareness*: each query must satisfy its TTFT constraint (τ_q); and *(3) Queue coupling*: concurrent queries compete for shared GPU resources, exacerbating HOL blocking under bursty arrivals.

Scheduler view. We use an SLO-aware scheduler that ranks queries using signals from Phases 1–2. (1) We derive an urgency signal from each query's remaining TTFT slack, making the scheduler deadline-aware (Challenge 2). (2) We normalize urgency by the estimated prefill cost, preventing expensive queries from dominating and addressing cost heterogeneity (Challenge 1). (3) We use this urgency-per-cost ordering to mitigate queue coupling (Challenge 3) by reducing HOL blocking and interleaving short urgent jobs with longer ones. Finally, we incorporate a difficulty signal to further ensure fairness, preventing hard queries from being systematically deprioritized.

Priority signals. We adopt a lightweight scheduling score that integrates cost-model estimates with signals from Phase 2.

SLO urgency. We define the SLO urgency of a query q as

$$U_{\text{slo}}(q) = \min\big(p_q/\tau_q,\ 1\big), \tag{13}$$

where $p_q = L_q/\mu_p$ denotes the estimated prefill time and τ_q is the TTFT deadline. When $U_{\text{slo}}(q)$ is close to 1, it indicates that prefill alone nearly exhausts the SLO budget.

Difficulty-aware amplification. We further amplify urgency using difficulty score from Phase 2:

$$U^*_{\text{slo}}(q) = U_{\text{slo}}(q) \cdot \varphi(S_{\text{diff}}(q)), \tag{14}$$

where $\varphi(\cdot)$ is a monotone scaling function; higher difficulty in-

Algorithm 1: DynaGraph Closed-Loop Pipeline

Input : Query stream $\{q\}$, corpus $\mathcal{D}$, SLO deadlines $\{\tau_q\}$
Output: Answer stream $\{Y_q\}$

1 $S_{\text{sys}} \leftarrow 0$; $\mathcal{G} \leftarrow \text{Construct}(\mathcal{D})$
2 **foreach** *incoming query* q **do**
  /* Phase 1: Retrieval (Section 4.2) */
3 $\mathcal{G}_q, \mathcal{F}_q, C_q \leftarrow \text{Retrieve}(q, \mathcal{G})$
  /* Phase 2: Adaptive Aggregation (Section 4.3) */
4 $S_{\text{diff}} \leftarrow \mathbf{w}^\top \tilde{\mathbf{f}}_q$ // difficulty (Eq.5)
5 **if** $u(t) > \theta_{\text{bp}}$ **then**
6 $K_{\text{eff}} \leftarrow K_{\min} + (K_{\max} - K_{\min})\,\phi(t)$ // backpressure (Eq.8)
7 **else**
8 $K_{\text{eff}} \leftarrow K_{\max}$ // quality-first
9 **end**
10 $K_q \leftarrow K_{\min} + \lfloor (K_{\text{eff}} - K_{\min})\, S_{\text{diff}} \rceil$ // budget (Eq.9)
11 $P_q, L_q \leftarrow \text{Assemble}(C_q, \mathcal{G}_q, K_q)$ // Selection (Eq.11)
  /* Phase 3: SLO-Aware Scheduling (Section 4.4) */
12 $U^*_{\text{slo}} \leftarrow U_{\text{slo}} \cdot \varphi(S_{\text{diff}})$ // urgency (Eq.14)
13 $\mathcal{P}(q) \leftarrow w_{\text{slo}} U^*_{\text{slo}} - w_{\text{cost}} C_{\text{prefill}}$ // priority (Eq.16)
14 Insert q with key $\mathcal{P}(q)$; $Y_q \leftarrow \text{ServeHighestPriority}()$
15 $u(t) \leftarrow \text{QueueUtilization}()$; $S_{\text{sys}} \leftarrow \mathbb{I}(u(t) > \theta_{\text{bp}})$
  // feedback (Eq.6)
16 **end**

creases the multiplier, boosting urgency and preventing hard queries from being systematically deprioritized.

Cost signal. To mitigate HOL blocking, we normalize prefill cost as

$$C_{\text{prefill}}(q) = L_q / L_{\text{ref}}, \tag{15}$$

where L_q is the prefill token length and L_{ref} is a fixed reference budget. Larger values indicate longer and more expensive prefills that are more likely to induce HOL blocking.

Priority function. We combine these signals into a unified score:

$$\mathcal{P}(q) = w_{\text{slo}} \cdot U^*_{\text{slo}}(q) - w_{\text{cost}} \cdot C_{\text{prefill}}(q), \tag{16}$$

where $w_{\text{slo}}, w_{\text{cost}} \geq 0$ are weights. The first term prioritizes urgent queries, while the second penalizes expensive ones, approximating urgency per unit cost with minimal overhead.

Scheduling policy. Given priority $\mathcal{P}(q)$, we adopt a priority-driven policy: at each step, the scheduler admits highest-$\mathcal{P}(q)$ queries to the next GPU batch (prefill), subject to resource constraints. This mitigates HOL blocking (vs. FCFS) and prevents large-K_q queries from monopolizing resources (vs. deadline-only scheduling).

System coupling. Phase 3 closes the loop with Phase 2 via utilization $u(t)$: when $u(t) > \theta_{\text{bp}}$, it feeds back to tighten budgets (Eqs. 8–9). This feedback acts only on the evidence budget: retriever expansion and retrieved candidates remain fixed at serving time, and realized prefill cost is determined by K_q rather than by the number of retrieved candidates. Thus, Phase 2 controls query cost while Phase 3 controls execution order, enabling dynamic workload adaptation.

Control boundary. DynaGraph operates at admission time: Phase 2 bounds prefill work for new requests, while Phase 3 schedules the resulting heterogeneous costs. Decode remains non-preemptive, and its length is not directly controlled. A long decode can delay later prefills and indirectly increase the pending-prefill backlog captured by $u(t)$, causing subsequent admissions to receive tighter budgets. Appendices E.3 and E.10 in [4] evaluate controlled evidence omission and a simultaneous load-and-difficulty burst, respectively. Preemptive decode mechanisms (*e.g.*, FastServe [52]) and output-length-aware admission caps are complementary; their integration is left to future work.

Starvation avoidance. Priority scheduling may starve low-priority queries. Under sustained congestion, backpressure (Eq. 8) compresses budgets toward $K_{\min}$, equalizing processing times and mitigating starvation. An optional aging mechanism can be adopted as a fairness safeguard; see Appendix B.5 in [4] for a bounded-wait discussion under explicit stability and bounded-service assumptions.

Note. (1) *Scheduling alone is insufficient*: with fixed budgets, even an optimal scheduler cannot reduce per-query cost. (2) *Budget adaptation alone is insufficient*: without scheduling, bursty arrivals still cause HOL blocking. (3) DynaGraph combines both: Phase 2 introduces cost heterogeneity, and Phase 3 exploits it via SLO-aware scheduling, jointly improving answer quality and SLO compliance. (4) DynaGraph requires a one-time calibration before serving: the hardware-dependent prefill throughput μ_{p} is profiled for each model–hardware pair, while the topology weights $\mathbf{w}$ are selected from query-grouped calibration data and fixed during serving. The backpressure trigger θ_{bp} is a dimensionless utilization threshold, not a hardware-specific throughput parameter; no continuous workload-specific retuning is performed on the serving path.

4.5 Algorithm and Guarantee

In this subsection, we present the DynaGraph algorithm that integrates Phases 1–3 into a closed-loop control system, analyze its complexity, and establish its theoretical guarantees.

Algorithm. As shown in Algorithm 1, DynaGraph consists of three phases: (1) Phase 1 (*Sensor*, lines 1–3) initializes S_{sys}, constructs $\mathcal{G}$ from $\mathcal{D}$ (line 1), and, for each incoming query q, retrieves the subgraph $\mathcal{G}_q$ and features $\mathcal{F}_q$ (line 3); (2) Phase 2 (*Controller*, lines 4–11) maps $\mathcal{F}_q$ to $S_{\text{diff}} \in [0, 1]$ (line 4), computes the load-dependent ceiling K_{eff} from utilization $u(t)$ (lines 6–8), sets $K_q \in [K_{\min}, K_{\text{eff}}]$ (line 10), and assembles the prompt P_q with length L_q (line 11); (3) Phase 3 (*Actuator*, lines 12–15) amplifies urgency via S_{diff} (line 12), computes priority $\mathcal{P}(q)$ (line 13), enqueues q, and dispatches the highest-priority query q' for service, emitting its answer $Y_{q'}$ to the output stream (line 14); it then updates feedback by measuring $u(t)$ and emitting backpressure S_{sys} when $u(t) > \theta_{\text{bp}}$ (line 15).

Complexity analysis. Algorithm 1 takes $O(|V_q|+|E_q| + |C_q| \cdot K_q + \log n)$ per-query overhead (excluding LLM inference), where $O(|V_q|+|E_q|)$ is for subgraph retrieval and feature extraction (Phase 1), $O(|C_q| \cdot K_q)$ is for diversity-aware evidence selection from $|C_q|$ candidates (Phase 2), and $O(\log n)$ is for priority-queue insertion and dispatch among n concurrent queries (Phase 3).

Guarantee. We establish the following guarantee for the joint optimization formulation.

Theorem 2: *Let $\mathcal{J}(\mathbf{K}, \pi)$ denote the QA-Goodput under budget vector $\mathbf{K}$ and serving order π. Let $\mathbf{K}^{\text{fix}}$ be the static budget (K_i=$K_{\max}$ for all i), π^{FCFS} the arrival-order schedule, and $(\mathbf{K}^*, \pi^*)$ a globally optimal solution to the joint problem (Eq. 3). Then, $\mathcal{J}(\mathbf{K}^*, \pi^*) \geq \max\big(\mathcal{J}(\mathbf{K}^*, \pi^{\text{FCFS}}), \mathcal{J}(\mathbf{K}^{\text{fix}}, \pi^*)\big)$.* □

Proof. Both $(\mathbf{K}^*, \pi^{\text{FCFS}})$ and $(\mathbf{K}^{\text{fix}}, \pi^*)$ are feasible solutions to the joint problem in Eq. 3. Since $(\mathbf{K}^*, \pi^*)$ is globally optimal, its objective value is no smaller than that of either feasible solution,

yielding the stated inequality. See Appendix B of [4] for details. □

Theorem 2 formalizes the complementarity observed in Table 1: neither adaptive budgets nor scheduling alone suffices, whereas an optimal joint solution dominates both. Additional results are provided in the Appendix B of [4], including scheduling approximation, queue stability, design rationale for exponential backpressure, and bounded starvation for DynaGraph and its components.

5 EXPERIMENTAL EVALUATION

Goal. We study five questions:

- **RQ1 (Overall performance):** Under concurrent load, how does DynaGraph compare with existing GraphRAG pipelines?
- **RQ2 (Serving strategy):** With retrieval fixed, how does DynaGraph compare with KV-cache backends, and what does the Phase 2–3 closed loop contribute?
- **RQ3 (Design choice ablation):** How do Phase 1 retrieval choices and adaptive-aggregation designs affect the quality–latency trade-off?
- **RQ4 (Robustness):** How robust is DynaGraph to request load and deployment knobs (*e.g.*, $K_{\min}$ and θ_{bp})?
- **RQ5 (Scalability):** Under fully live retrieval, does DynaGraph remain effective and generalize across LLMs and corpus scales?

Explanation. (1) RQ1 compares DynaGraph with representative GraphRAG pipelines under concurrent load. (2) RQ2 fixes retrieval (*i.e.*, treats C_q as given) and compares KV-cache backends to quantify the joint benefit of adaptive budgeting and scheduling. (3) RQ3 ablates Phase 1 retrieval choices and Phase 2's mapping from graph structure and runtime load to per-query budgets, including the exponential decay in Eq. (8). (4) RQ4 varies workload intensity and system parameters. (5) RQ5 tests end-to-end effectiveness under live embedding, retrieval, and reranking.

Evaluation protocol. (1) RQ1 uses method-specific evidence prepared before replay, so TTFT measures generation-serving latency; AdaptiveRAG retains its online router and route-induced LLM calls (Appendix D.4 of [4]). (2) Except for RQ3's controlled Phase 1 candidate-source study, RQ2–RQ4 fix C_q and vary only budgeting, ranking, and/or scheduling to isolate Phases 2 and 3. (3) RQ5 evaluates DynaGraph with live query encoding, retrieval and reranking.

We further study context selection (RQ6) and phase ablation (RQ7) in Appendix E of [4].

5.1 Setup

Datasets. We evaluate on three established multi-hop QA benchmarks: 2WikiMultihopQA [21] (2Wiki; 12,576 queries; multi-path, bridge-entity reasoning), HotpotQA [58] (7,405 queries; cross-document reasoning), and MuSiQue [48] (2,417 queries; compositional reasoning up to 4 hops). Following the protocol of HippoRAG [19] and subsequent works [22, 46, 63], we sample 1,000 queries from each validation set, using their associated supporting and distractor passages as the retrieval corpus $\mathcal{D}$ (approximately 11K passages per dataset). For scalability analysis (RQ5), we additionally evaluate on the full 2Wiki validation set (~57K passages).

Models. We use two primary LLMs spanning distinct architectural families: Qwen3-4B [54] (pure attention; single RTX 4090) and Qwen3.5-4B [42] (hybrid Mamba; 2×RTX 4090 with tensor parallelism, TP=2). We additionally include Qwen2.5-7B [55] in RQ2 to evaluate performance under moderate serving load. For retrieval, we use Qwen3-Embedding-0.6B [65] for dense encoding and bge-reranker-v2-m3 [11] for cross-encoder reranking.

Infrastructure. All experiments are conducted on 2×NVIDIA RTX 4090 GPUs (24 GB each) with 128 GB of RAM. We use vLLM [32] for LLM serving. Unless otherwise specified for a given RQ, workloads follow the Zipfian popularity model from Section 3.1 (α=1.0, λ=30 QPS; 3,000 total requests). Please see Appendix C in [4] for more infrastructure details.

Metrics. We evaluate along four complementary dimensions: (1) *Answer quality*, measured by token-level F_1 on deduplicated unique queries to avoid popularity bias under Zipf replay [48, 58]; (2) *Serving latency*, measured by Time-to-First-Token (TTFT) at p50/p95, the primary latency metric in LLM serving systems [32, 67]; (3) *Service reliability*, measured by SLO attainment, *i.e.,* the fraction of requests completed within a deadline τ [2, 67]; and (4) *QA-Goodput*, $\mathcal{J}_\lambda$ (Eq. 2), which weights each on-time request by its F_1 score, jointly capturing accuracy and timeliness in a single metric. $\mathcal{J}_\lambda$ is computed per-request to reflect actual serving conditions. We report $\mathcal{J}_\lambda$ at $\tau \in \{200, 500, 2000\}$ ms to cover strict to relaxed SLO regimes; all latency values are in ms unless stated otherwise.

Baselines. We implemented DynaGraph in Python and compared against the following baselines with released code: (1) *GraphRAG-based methods:* (a) SiReRAG [63] (dual hierarchical indexing), (b) KET-RAG [22] (KG skeleton with a bipartite keyword–entity graph), (c) ClueRAG [46] (query-driven iterative traversal with reranking), and (d) HippoRAG [19] (lightweight PPR-based graph propagation); (2) *KV-cache-based serving methods* (for RQ2): (e) CacheBlend [59] (chunk-level KV offloading), and (f) DepCache [60] (sparse dependency attention for graph-path prefix sharing); (3) *an adaptive RAG method:* (g) AdaptiveRAG [26] (query-complexity-based routing among retrieval strategies); and (4) *a DynaGraph variant:* (h) VanillaGRAG, which directly uses the Phase 1 output without the optimizations in Phases 2 and 3.

For targeted diagnostics, we also include Adaptive-K [47], RAG-CSM [15] and CompSelect [64] in the estimator/router audits and context-selection studies.

Within each comparison, all methods share the stated underlying LLM, hardware, workload generator, and evaluated request set. We use released implementations and reported settings where available and document method-required serving backends, adaptations, and online-cost accounting in Appendix D of [4].

5.2 RQ1: Overall Performance

We compare DynaGraph against 4 representative GraphRAG pipelines (SiReRAG, KET-RAG, ClueRAG and HippoRAG) and AdaptiveRAG. As shown in Table 3, we find the following.

(1) DynaGraph achieves the best answer quality across all datasets (F_1: 0.787/0.769/0.471), outperforming the strongest baseline by 9–74%. The gain is largest on MuSiQue (+0.200 over HippoRAG and +0.062 over ClueRAG), where compositional multi-hop reasoning requires bridging evidence across entity clusters. This is because DynaGraph preserves such dispersed evidence via coverage-aware assembly (Section 4.3.3), whereas single-strategy

Table 3: End-to-end system comparison under online serving. Qwen3.5-4B, 1000 unique queries, Zipf@30 qps, and 3000 total requests. $\mathcal{J} = \frac{1}{N}\sum_i F_1^{(i)} \cdot \mathbb{I}[\text{TTFT}_i \le \tau]$. $\mathcal{J}^{\text{soft}}$ grants full credit before τ and linearly decaying credit from τ to 2τ.

Method	F1	Acc	Toks	p50 (ms)	SLO@500	SLO@200	$\mathcal{J}$@500	$\mathcal{J}$@200	$\mathcal{J}^{\text{soft}}$@500
2WikiMultihopQA									
SiReRAG	.721	.728	6760	8629	0.0%	0.0%	0.0%	0.0%	0.0%
KET-RAG	.470	.438	11833	19255	0.0%	0.0%	0.0%	0.0%	0.0%
ClueRAG	.637	.644	1165	788	6.8%	0.1%	4.8%	0.1%	28.8%
HippoRAG	.601	.673	961	563	34.8%	0.2%	22.1%	0.1%	47.9%
AdaptiveRAG	.305	.536	6911	393	69.8%	11.9%	20.1%	3.8%	28.2%
DynaGraph	**.787**	**.841**	2391	**220**	**94.9%**	**45.5%**	**72.8%**	**37.5%**	**74.6%**
HotpotQA									
SiReRAG	.570	.483	5482	6346	0.0%	0.0%	0.0%	0.0%	0.0%
KET-RAG	.603	.524	11613	19144	0.0%	0.0%	0.0%	0.0%	0.0%
ClueRAG	.512	.455	709	725	5.8%	0.1%	3.3%	0.0%	27.4%
HippoRAG	.499	.535	1202	509	47.2%	1.3%	21.9%	0.7%	40.8%
AdaptiveRAG	.282	.429	5810	461	56.7%	12.2%	13.5%	3.0%	24.3%
DynaGraph	**.769**	**.716**	2244	**224**	**95.5%**	**40.3%**	**53.3%**	**21.5%**	**54.9%**
MuSiQue									
SiReRAG	.427	.266	6712	7751	0.0%	0.0%	0.0%	0.0%	0.0%
KET-RAG	.301	.152	11909	19060	0.0%	0.0%	0.0%	0.0%	0.0%
ClueRAG	.409	.250	904	715	6.8%	0.2%	3.1%	0.0%	23.3%
HippoRAG	.271	.265	1282	481	53.9%	1.0%	14.8%	0.1%	23.4%
AdaptiveRAG	.139	.344	7786	552	42.5%	4.6%	14.4%	1.6%	29.4%
DynaGraph	**.471**	**.507**	2088	**173**	**97.7%**	**63.6%**	**47.6%**	**32.8%**	**47.6%**

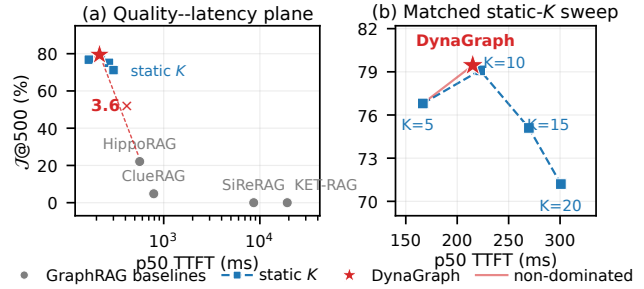


Figure 6: SLO-aware quality–latency operating points (2Wiki, Qwen3.5-4B, Zipf@30 QPS). Static K replaces adaptive budgeting only; the red line marks the empirical frontier over evaluated configurations.

retrievers discard it (HippoRAG) or fail to prioritize it (ClueRAG).

(2) DynaGraph has the lowest median TTFT (220/224/173 ms), despite using longer contexts than lightweight baselines such as HippoRAG and ClueRAG. The large fixed contexts of SiReRAG and KET-RAG (5–12k tokens) cause extreme latency (p50 up to 19.3 s) and complete SLO failure, while ClueRAG and HippoRAG lack scheduling coordination despite moderate context sizes, yielding higher latency (481–788 ms) and lower SLO attainment (6.8–53.9%). In contrast, DynaGraph shortens most easy-query contexts via adaptive budgeting and prioritizes low-cost queries via SLO-aware scheduling, reducing median prefill cost and queueing delay.

(3) DynaGraph achieves the best *QA-Goodput*, reaching $\mathcal{J}$@500 of 72.8%/53.3%/47.6%, outperforming AdaptiveRAG by 3.3×–4.0× (20.1%/13.5%/14.4%). Unlike AdaptiveRAG's iterative retrieval–generation with multiple LLM calls per request, DynaGraph budgets and schedules evidence before a single generation, substantially reducing serving costs under the shared load. Under the stricter τ=200 ms SLO, DynaGraph reaches $\mathcal{J}$@200 of 37.5%/21.5%/32.8%, while all baselines remain below 1.3%. When late responses receive partial credit, DynaGraph also achieves the highest soft time utility: $\mathcal{J}_\lambda^{\text{soft}}$@500 reaches 74.6%/54.9%/47.6%, exceeding the strongest baseline by 26.7/14.1/18.2 percentage points, respectively. Thus, DynaGraph delivers high-quality and timely answers under hard/-soft latency utilities via joint evidence budgeting and scheduling.

To test whether these gains reflect a better SLO-aware quality–latency operating point, Figure 6 plots $\mathcal{J}$@500 against median TTFT. Alongside the GraphRAG baselines, it includes matched static-budget points obtained by replacing only adaptive budgeting with $K \in \{5, 10, 15, 20\}$ in a separate matched same-stack replay. The sweep is non-monotone: the best fixed setting (K=10) reaches $\mathcal{J}$@500 = 79.06% at 222.3 ms, whereas DynaGraph reaches 79.45% at 215.1 ms. Thus, DynaGraph lies on the empirical frontier over the evaluated configurations: adaptive budgeting and backpressure match or slightly improve the best measured fixed-budget point without preselecting its budget, placing the closed-loop design in a favorable empirical quality–latency region.

5.3 RQ2: Serving Strategy Comparison

We evaluate the serving strategy enabled by the closed-loop interaction between Phases 2 and 3 by isolating the serving layer. Specifically, we fix the retriever (Phase 1) and compare DynaGraph against variants that replace Phases 2 and 3 with KV-cache-based methods (CacheBlend and DepCache) or a no-optimization baseline (VanillaGRAG). For fair comparison, all methods receive the same Phase 1 retrieval output (~30k tokens).

Table 4 shows the following. (1) DynaGraph achieves the highest $\mathcal{J}$@500 on Qwen3-4B and Qwen2.5-7B (78.6%/64.8%). It matches DepCache's lowest Qwen3-4B latency (p50: 143 vs. 141 ms) while providing 2.4×/1.4× higher F1, yielding a better quality–latency trade-off. (2) By forwarding the Phase 1 output unpruned, VanillaGRAG establishes the cost upper bound but incurs prohibitive latency (p50: 45 s/4.9 s) and near-zero QA-Goodput ($\mathcal{J}$@500≤1.4%), confirming the need for prompt-length control. (3) CacheBlend's per-chunk KV reuse improves prefill throughput without shortening the prompt, yielding high latency (p50: 18.3 s/11.0 s) and 0% SLO@500. It also loses cross-document dependencies, reducing F1 to 0.667 (−14%); improving μ_p alone cannot offset an oversized context. (4) DepCache achieves low latency (p50: 141 ms; 99.5% SLO@500) but lower quality (F1: 0.308/0.427; $\mathcal{J}$@500: 30.7%/41.2%). Its attention over graph-structured token pairs misses document- and chunk-level evidence, causing incomplete reasoning and showing that sparse attention can break multi-hop integration.

By reducing prompt length *before* inference via adaptive K while preserving all evidence types under full attention, DynaGraph achieves F_1 of 0.752/0.602 and the highest $\mathcal{J}$@200 of 61.1%/47.5%. Thus, the Phase 2–3 closed loop jointly controls cost and execution order, improving QA-Goodput under realistic serving constraints.

We further ablate the contributions of Phases 2 and 3 individually on Qwen3.5-4B (see Appendix E.4 in full version [4] for details). We find that neither phase alone can achieve both high SLO attainment and high QA-Goodput, consistent with the observations in Section 3.2. This further justifies the need for our joint design that couples adaptive budgeting (Phase 2) with SLO-aware scheduling.

5.4 RQ3: Design Choice Ablation

We analyze key design choices in DynaGraph's retrieval and controller stages. We study (1) the decay shape of the budget ceiling $K_{\text{eff}}(t)$ under congestion (Eq. (8)), (2) the cost-aware difficulty estimator $S_{\text{diff}}(q)$ (Eq. (5)), (3) the contribution of different Phase 1 candidate sources (see Appendix E.1 in [4]), and (4) sensitivity of the tar-

Table 6: Sensitivity to Hop-1 retrieval depth (2Wiki, Qwen3-4B, Zipf@30 QPS).

Hop-1 M	F1	Isolated Phase 1 time	Toks	p50/p95 (ms)	$\mathcal{J}$@200	$\mathcal{J}$@500
5	**0.7859**	364.4 ms	2461	237 / 543	35.42%	**78.41%**
10	0.7603	698.5 ms	2388	232 / 517	35.33%	78.00%
20	0.7444	1.37 s	2512	213 / 481	**39.98%**	77.25%
50	0.7330	3.37 s	2613	225 / 536	36.90%	76.23%
100	0.7377	6.71 s	2645	226 / 545	38.46%	76.05%
200	0.7429	13.40 s	2717	225 / 528	37.89%	76.98%

Table 4: Serving baseline comparison with fixed retriever. 2Wiki, 1000q×3000, Zipf@30 qps.

Method	F1↑	Acc↑	p50 (ms)↓	p95 (ms)↓	SLO@500↑	$\mathcal{J}$@500↑	$\mathcal{J}$@200↑
Qwen3-4B							
VanillaGRAG	**.777**	**.817**	45053	85912	0.2%	0.2%	0.0%
CacheBlend [59]	.667	.725	18298	34090	0.0%	0.0%	0.0%
DepCache [60]	.308	.466	141	344	**99.5%**	30.7%	21.3%
DynaGraph	.752	.797	**143**	**339**	97.9%	**78.6%**	**61.1%**
Qwen2.5-7B							
VanillaGRAG	.555	.587	4885	10827	2.8%	1.4%	0.7%
CacheBlend [59]	.564	.595	11035	12655	0.0%	0.0%	0.0%
DepCache [60]	.427	.457	213	467	**96.3%**	41.2%	19.8%
DynaGraph	**.602**	**.621**	**144**	**433**	96.2%	**64.8%**	**47.5%**

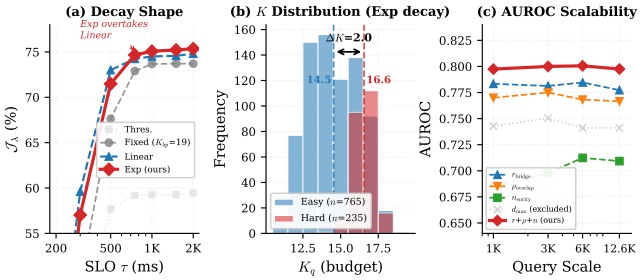


Figure 7: Phase 2 design ablation (2Wiki, Qwen3.5-4B, λ=30). (a) Exp decay achieves the best F1 while bounding tail latency. (b) Hard/Easy K separation under backpressure. (c) Three features (r+ρ+n) achieve stable AUROC ≈ 0.80; d_{max} excluded.

Table 5: Matched-budget estimator audit (2Wiki).

Estimator	Spearman ρ(score, K^*)	AUROC	AUPRC	Selector p50 (ms)
RAG-CSM	.254	.638	.307	25.33
score-ratio heuristic (adaptive-K)	.013	.457	.207	<.001
structural S_{diff}	.260	.656	.326	<.001
retrieval guardrail	.201	.584	.248	.008
structural + retrieval guardrail	**.375**	**.695**	**.391**	.009

get quality–latency objective to the static Hop-1 retrieval depth M.

Impact of decay shape of $K_{eff}(t)$. We compare four strategies: (a) *Exponential* uses the load-dependent decay in Eq. (8); (b) *Linear* decreases $K_{\text{eff}}(t)$ linearly with the normalized excess utilization $(u(t) - \theta_{\text{bp}})/(1 - \theta_{\text{bp}})$; (c) *Fixed* sets $K_{\text{eff}}(t) = K_{\text{bp}}$ to a constant reduced ceiling once congestion is triggered; and (d) *Threshold* directly sets $K_{\text{eff}}(t) = K_{\min}$ whenever $u(t) > \theta_{\text{bp}}$.

As shown in Figure 7(a), exponential decay offers the best overall quality–latency trade-off, achieving the highest F1 (0.767) and $\mathcal{J}$@2s (75.4%) with a larger average budget (avg K=12.5, +24% over Linear) that preserves hard-query context under sustained load. Linear slightly improves $\mathcal{J}$@200 (37.8% vs. 35.7%) but over-compresses budgets, lowering F1 to 0.761 and weakening difficulty differentiation. Fixed cannot absorb load spikes (p99 up to 15,687 ms), while Threshold collapses all queries to $K_{\min}$ and F1 to 0.605. Thus, smooth load-aware decay balances quality and latency across SLO regimes.

We further examined its interaction with backpressure from Phase 3 (Figure 7(b)). (a) Exponential decay reduces avg K from 20.0 to 12.5 (−37.5%) while preserving clear separation between hard and easy queries (83% vs. 72.5% of $K_{\max}$), with no queries collapsed to $K_{\min}$. (b) In contrast, linear and threshold strategies compress all queries to similar budgets, eliminating the difficulty signal. This pattern generalizes across datasets (avg K≈12.4–12.6) while maintaining high SLO@2s (≥98.6%). These results highlight the robustness of exponential decay under dynamic backpressure.

Impact of topological difficulty features. As shown in Figure 7(c), combining the three Phase 1 signals (r_{bridge}, ρ_{overlap}, and n_{entity}) performs best, maintaining AUROC ≈ 0.80 from 1K to 12.6K queries. Individually, r_{bridge} and ρ_{overlap} slightly degrade with graph density, while n_{entity} becomes more discriminative; their combination remains stable. Adding $d_{\max}$ (the maximum node degree in the query-induced subgraph $\mathcal{G}_q$) improves AUROC by less than 1 pp, indicating diminishing returns. Thus, the three features suffice for effective difficulty estimation.

Difficulty-estimator audit. Beyond feature ablation, we audit lightweight signals for false "easy" cases using per-query oracle budgets K^* reconstructed from the static-K sweep on 2Wiki. As shown in Table 5, the pre-execution S_{diff} without the guardrail achieves Spearman ρ=0.260, AUROC= 0.656, and AUPRC= 0.326, slightly outperforming RAG-CSM (0.254/0.638/0.307) with negligible latency (< 0.001 ms vs. 25.33 ms). False "easy" cases can still arise when a sparse query-induced graph contains flat or conflicting retrieved evidence. The retrieval-ambiguity guardrail complements topology with retrieval-score dispersion, improving the audit to ρ=0.375/AUROC= 0.695/AUPRC= 0.391 with only 0.009 ms selector latency. Thus, it improves estimator–oracle alignment in the matched-budget audit, providing a low-overhead optional check without placing a trained selector on the common serving path.

Sensitivity to Hop-1 retrieval depth. We test the target quality–latency objective under static depths M (*i.e.*, the K_1 in Section 4.2) over {5, 10, 20, 50, 100, 200}, fixing the Phase 2 controller, Phase 3 scheduler, corpus, reader and arrival trace. For each M, we rebuild and rerank its candidates over the full trace to profile Phase 1; all other metrics use the resulting artifacts preloaded. Table 6 shows that the default M=5 performs best, with the highest F1 (0.7859) and $\mathcal{J}$@500 (78.41%). Increasing M to 20 improves the stricter SLO, reducing admitted p95 TTFT from 543 to 481 ms and increasing $\mathcal{J}$@200 from 35.42% to 39.98%. However, it lowers F1 by 0.0415 and $\mathcal{J}$@500 by 1.16 points, while increasing isolated Phase 1 time from 364 ms to 1.37 s (and 13.40 s at M=200). Beyond M=20, prompt tokens grow without improving answer quality or QA-Goodput. Thus, larger M mainly adds Phase 1 overhead without benefiting the 500-ms objective; we fix M=5 and confine backpressure to Phases 2–3, where it adapts the downstream budget K_q.

These results validate the Phase 2 controller design: smooth decay and topology-aware difficulty estimation are important for preserving informative context under dynamic system load.

5.5 RQ4: Robustness

We evaluate the robustness of DynaGraph through three sensitivity studies and a burst stress test: (1) sensitivity to the minimum query budget $K_{\min}$, (2) sensitivity to the backpressure activation threshold θ_{bp}, (3) sensitivity under increasing request-load rate λ, and (4) robustness under simultaneous high load and hard-query arrivals.

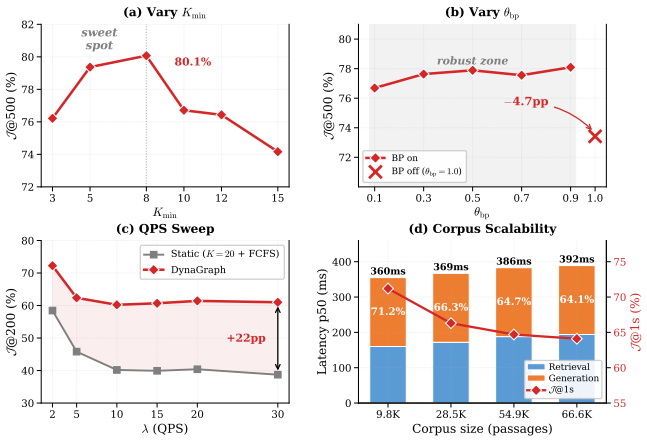


Figure 8: Robustness. (a) $K_{\min}$ sweep. (b) θ_{bp} sweep; × marks BP disabled. (c) DynaGraph vs Static under increasing load. (d) E2E latency breakdown and $\mathcal{J}$@1s under corpus growth.

Varying minimum budget size $K_{\min}$. We evaluate the performance of DynaGraph by varying $K_{\min}$ from 3–15. As shown in Figure 8(a), performance improves as $K_{\min}$ increases from 3 to 8, reaching a peak $\mathcal{J}\lambda$@500 of 80.1%, and then gradually declines for larger values. This is because a small $K_{\min}$ under-allocates evidence for hard queries, while an overly large $K_{\min}$ reduces flexibility and pushes latency toward the SLO boundary. We find that DynaGraph performs well when $K_{\min} \in [5, 10]$.

Varying backpressure activation threshold θ_{bp}. We evaluate the performance of DynaGraph by varying θ_{bp} from 0.1–1.0. As shown in Figure 8(b), performance remains stable when $\theta_{bp} \in [0.1, 0.9]$, with $\mathcal{J}\lambda$@500 varying within 1.4%, and drops by 4.4pp when $\theta_{bp} = 1.0$. This is because moderate θ_{bp} enables timely activation of backpressure under congestion, whereas disabling it reverts to a static allocation that fails under strict SLOs. We find that DynaGraph is robust to θ_{bp} within a wide range.

Varying request load ratio λ. We evaluate the performance of DynaGraph by varying λ from 2–30. As shown in Figure 8(c), DynaGraph maintains stable performance (60–72% $\mathcal{J}_\lambda$@200), while the static baseline (i.e., the phase-silo configuration in Section 3.1) drops significantly from 59% to 39%. This is because difficulty-aware budget allocation provides a strong baseline, while backpressure further mitigates congestion as load increases. We observe that DynaGraph remains stable under increasing workload intensity.

We also evaluate calibration-sample sufficiency, topology-weight sensitivity, GPU energy, token-level inter-token latency, and post-spike controller recovery, and conduct a controlled evidence-omission diagnostic. We further study simultaneous load-and-difficulty bursts, showing that DynaGraph adapts its budgets under peak load while supporting configurable quality–latency trade-offs. See Appendices E.8, E.3, E.9 and E.10 in [4] for details.

5.6 RQ5: Scalability

We evaluate the scalability of DynaGraph along two key dimensions: (1) graph and query scalability under corpus growth and fully live pipelines, and (2) model scalability across different LLM sizes.

Graph and query scalability. Figure 8(d) shows that DynaGraph scales with corpus size while maintaining strong fully live end-to-end performance. As the corpus grows 7× from 9.8K to 66.6K passages, end-to-end p50 latency increases only 9% (360⟶392 ms), while $\mathcal{J}$@1s decreases moderately (71.2%⟶64.1%). Its fully online F_1 also remains comparable to the pre-computed setting (*e.g.,* 0.801 vs. 0.787 on 2Wiki). This scalability follows because Phase 1 bounds retrieval through two-hop expansion and fixed-size reranking, while Phase 2 processes a fixed-size candidate set, making cost largely insensitive to corpus growth. Thus, DynaGraph scales to larger corpora without prohibitive latency growth and remains competitive under dynamic retrieval and evolving data.

See Appendix E.6 in [4] for detailed cross-model results.

6 RELATED WORK

We group existing work into three categories.

Graph-based retrieval for multi-hop QA. Hierarchical methods such as RAPTOR [45] and GraphRAG [16] organize corpora into trees or communities to surface long-range evidence. Entity/relation-centric methods, including HippoRAG [19], SiReRAG [63], KET-RAG [22], and Clue-RAG [46], use explicit graph indices for multi-hop expansion through dual-perspective indexing, multi-granular graphs, or query-driven traversal. Although improving evidence coverage and reasoning faithfulness, they typically use fixed retrieval budgets and ignore serving-time latency and throughput constraints.

RAG serving and caching. General LLM serving systems, including vLLM [32], SGLang [66], and DistServe [67], improve memory management and scheduling but remain retrieval-agnostic. RAG-oriented systems, such as RAGO [28], METIS [44], RAGCache [29], CacheBlend [59], and DepCache [60], reduce prefill overhead via configuration adaptation or KV-cache reuse. However, they optimize execution after evidence selection rather than controlling evidence budgets or adapting retrieval to instantaneous load.

Adaptive context selection. AdaptiveRAG [26] and Self-RAG [5] route queries by estimated complexity; RECOMP [53] and LongLLMLingua [27] compress post-retrieval prompts; and Adaptive-k [47] and CompSelect [64] predict query-specific budgets. These methods expose the waste of fixed-K prompting but mainly rely on textual similarity or learned surrogates and rarely integrate serving backpressure.

How DynaGraph differs. DynaGraph unifies retrieval, budget selection and serving. It uses lightweight graph-structural signals to estimate difficulty, controls evidence proactively rather than only compressing/caching after retrieval, and couples this control with load-aware scheduling to jointly optimize answer quality and timeliness.

7 CONCLUSION

We present DynaGraph, a closed-loop GraphRAG framework that jointly optimizes evidence selection and serving under SLO constraints. Built on the proposed QA-Goodput metric, our preliminary analysis reveals that fixed-budget retrieval and phase-isolated pipelines break down under cost heterogeneity and bursty arrivals, as they fail to align retrieval decisions with serving-time dynamics. DynaGraph addresses this limitation through a unified, feedback-driven design: (a) a topology-based difficulty estimator that captures query complexity, (b) a backpressure-aware budget controller that adapts context under system load, and (c) an SLO-aware scheduler that prioritizes queries based on urgency and estimated cost.

REFERENCES

[1] Shubham Agarwal, Sai Sundaresan, et al. 2025. Cache-craft: Managing chunk-caches for efficient retrieval-augmented generation. *Proceedings of the ACM on Management of Data (SIGMOD)* 3, 3 (2025), 1–28.

[2] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav Gulavani, Alexey Tumanov, and Ramachandran Ramjee. 2024. Taming Throughput-Latency tradeoff in LLM inference with Sarathi-Serve. In *18th USENIX symposium on operating systems design and implementation (OSDI 24)*. 117–134.

[3] Aisha Alansari and Hamzah Luqman. 2026. Large language models hallucination: A comprehensive survey. *Computer Science Review* 61 (2026), 100970.

[4] Anonymous. 2026. DynaGraph: Code, Artifacts, and Full Version. https://anonymous.4open.science/r/DynaGraph-Releasee-0496

[5] Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. 2024. Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection. In *The Twelfth International Conference on Learning Representations (ICLR)*.

[6] Betsy Beyer, Chris Jones, Jennifer Petoff, and Niall Richard Murphy. 2016. *Site Reliability Engineering: How Google Runs Production Systems*. O'Reilly Media, Inc.

[7] Angela Bonifati, Wim Martens, John Timm, et al. 2017. An Analytical Study of Large SPARQL Query Logs. *Proceedings of the VLDB Endowment* 11, 2 (2017), 149–161.

[8] Lee Breslau, Pei Cao, Li Fan, Graham Phillips, and Scott Shenker. 1999. Web caching and Zipf-like distributions: Evidence and implications. In *Proceedings of IEEE INFOCOM*. IEEE.

[9] Niv Buchbinder and Joseph Naor. 2009. *The design of competitive online algorithms via a primal-dual approach*. Now Publishers.

[10] Deeparnab Chakrabarty, Yunhong Zhou, and Rajan Lukose. 2008. Online knapsack problems. In *Workshop on Internet and Network Economics (WINE)*.

[11] Jianlv Chen, Shitao Xiao, Peitian Zhang, Kun Luo, Defu Lian, and Zheng Liu. 2024. BGE M3-Embedding: Multi-Lingual, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation. arXiv:2402.03216 [cs.CL]

[12] Alla Chepurova, Aydar Bulatov, Mikhail Burtsev, and Yuri Kuratov. 2026. Wikontic: Constructing Wikidata-Aligned, Ontology-Aware Knowledge Graphs with Large Language Models. In *EACL*. 8304–8319. https://doi.org/10.18653/v1/2026.eacl-long.388

[13] Yingnong Dang, Qingwei Lin, and Peng Huang. 2019. AIOps: Real-World Challenges and Research Innovations. In *Proceedings of the 41st IEEE/ACM International Conference on Software Engineering: Companion (ICSE-Companion)*. 4–5.

[14] George B. Dantzig. 1957. Discrete-Variable Extremum Problems. *Operations Research* 5, 2 (1957), 266–288.

[15] Jiale Deng, Yanyan Shen, Ziyuan Pei, Youmin Chen, and Linpeng Huang. 2025. Influence Guided Context Selection for Effective Retrieval-Augmented Generation. In *Advances in Neural Information Processing Systems (NeurIPS)*.

[16] Darren Edge et al. 2024. From Local to Global: A Graph RAG Approach to Query-Focused Summarization. *arXiv preprint arXiv:2404.16130* (2024).

[17] Teofilo F. Gonzalez. 1985. Clustering to Minimize the Maximum Intercluster Distance. *Theoretical Computer Science* 38 (1985), 293–306.

[18] Arpan Gujarati, Reza Karimi, Safya Alzayat, Wei Hao, Antoine Kaufmann, Ymir Vigfusson, and Jonathan Mace. 2020. Serving DNNs like Clockwork: Performance Predictability from the Bottom Up. In *Proceedings of the 14th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*.

[19] Bernal Jiménez Gutiérrez, Yiheng Shu, Yu Gu, Michihiro Yasunaga, and Yu Su. 2024. HippoRAG: Neurobiologically Inspired Long-Term Memory for Large Language Models. In *Advances in Neural Information Processing Systems (NeurIPS)*.

[20] Joseph L Hellerstein, Yixin Diao, Sujay Parekh, and Dawn M Tilbury. 2004. *Feedback control of computing systems*. John Wiley & Sons.

[21] Xanh Ho, Anh-Khoa Duong Nguyen, Saku Sugawara, and Akiko Aizawa. 2020. Constructing A Multi-Hop QA Dataset for Comprehensive Evaluation of Reasoning Steps. In *COLING*.

[22] Yiqian Huang, Shiqi Zhang, and Xiaokui Xiao. 2025. Ket-rag: A cost-efficient multi-granular indexing framework for graph-rag. In *KDD*. 1003–1012.

[23] Ihab F. Ilyas, Theodoros Rekatsinas, Vishnu Konda, Jeffrey Pound, Xiaoguang Qi, and Mohamed Soliman. 2022. Saga: A Platform for Continuous Construction and Serving of Knowledge at Scale. In *SIGMOD*. 2259–2272.

[24] Isaacus. 2025. semchunk: A fast, lightweight and easy-to-use Python library for splitting text into semantically meaningful chunks. https://github.com/isaacus-dev/semchunk

[25] E. Douglas Jensen, C. Douglass Locke, and Hideyuki Tokuda. 1985. A Time-Driven Scheduling Model for Real-Time Operating Systems. In *RTSS*. 112–122.

[26] Soyeong Jeong, Jinheon Baek, Sukmin Cho, Sung Ju Hwang, and Jong C. Park. 2024. Adaptive-RAG: Learning to Adapt Retrieval-Augmented Large Language Models through Question Complexity. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*. 7036–7050.

[27] Huiqiang Jiang, Qianhui Wu, Xufang Luo, Dongsheng Li, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. 2024. LongLLMLingua: Accelerating and Enhancing LLMs in Long Context Scenarios via Prompt Compression. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*.

[28] Wenqi Jiang et al. 2025. RAGO: Systematic Performance Optimization for Retrieval-Augmented Generation Serving. In *ISCA*.

[29] Chao Jin, Zili Zhang, Xuanlin Jiang, Fangyue Liu, Shufan Liu, Xuanzhe Liu, and Xin Jin. 2025. Ragcache: Efficient knowledge caching for retrieval-augmented generation. *ACM Transactions on Computer Systems* 44, 1 (2025), 1–27.

[30] Kostis Kaffes, Timothy Chong, Jack T. Humphries, Adam Belay, David Mazières, and Christos Kozyrakis. 2019. Shinjuku: Preemptive Scheduling for μsecond-scale Tail Latency. In *Proceedings of the 16th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*.

[31] Richard M. Karp. 1972. Reducibility among Combinatorial Problems. In *Complexity of Computer Computations*. Plenum Press, 85–103.

[32] Woosuk Kwon et al. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *SOSP*.

[33] Patrick Lewis et al. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *NeurIPS*.

[34] Yihan Li, Xiyuan Fu, Ghanshyam Verma, Paul Buitelaar, and Mingming Liu. 2025. Mitigating Hallucination in Large Language Models (LLMs): An Application-Oriented Survey on RAG, Reasoning, and Agentic Systems. *arXiv preprint arXiv:2510.24476* (2025).

[35] Zhang Li et al. 2026. OntoKG: Ontology-Aware Knowledge Graph Construction from Unstructured Corpora. *arXiv preprint arXiv:2604.02618* (2026).

[36] Minghua Ma, Zheng Yin, Shenglin Zhang, Sheng Wang, Christopher Zheng, Xinhao Jiang, Hanwen Hu, Cheng Luo, Yilin Li, Nengjun Qiu, Feifei Li, Changcheng Chen, and Dan Pei. 2020. Diagnosing Root Causes of Intermittent Slow Queries in Cloud Databases. *Proceedings of the VLDB Endowment (PVLDB)* 13, 8 (2020), 1176–1189. https://doi.org/10.14778/3389133.3389136

[37] Costas Mavromatis and George Karypis. 2024. GNN-RAG: Graph Neural Retrieval for Large Language Model Reasoning. *arXiv preprint arXiv:2405.20139* (2024).

[38] Sean P. Meyn and Richard L. Tweedie. 2009. *Markov Chains and Stochastic Stability* (2nd ed.). Cambridge University Press.

[39] Michael Neely. 2010. *Stochastic network optimization with application to communication and queueing systems*. Morgan & Claypool Publishers.

[40] Zhaojian Niu et al. 2026. TokenPowerBench: Benchmarking the Power Consumption of LLM Inference. In *AAAI*.

[41] Shirui Pan, Linhao Luo, Yufei Wang, Chen Chen, Jiapu Wang, and Xindong Wu. 2024. Unifying Large Language Models and Knowledge Graphs: A Roadmap. *IEEE Transactions on Knowledge and Data Engineering* (2024).

[42] Qwen Team. 2026. Qwen3.5: Towards Native Multimodal Agents. https://qwen.ai/blog?id=qwen3.5

[43] Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev, and Percy Liang. 2016. Squad: 100,000+ questions for machine comprehension of text. In *Proceedings of the 2016 conference on empirical methods in natural language processing*. 2383–2392.

[44] Siddhant Ray, Rui Pan, Zhuohan Gu, Kuntai Du, Shaoting Feng, Ganesh Ananthanarayanan, Ravi Netravali, and Junchen Jiang. 2025. METIS: Fast Quality-Aware RAG Systems with Configuration Adaptation. In *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles (SOSP)*. 606–622.

[45] Parth Sarthi, Salman Abdullah, Aditi Tuli, Shubh Khanna, Anna Goldie, and Christopher D. Manning. 2024. RAPTOR: Recursive Abstractive Processing for Tree-Organized Retrieval. In *Proceedings of the 12th International Conference on Learning Representations (ICLR)*.

[46] Yaodong Su, Yixiang Fang, Yingli Zhou, Quanqing Xu, and Chuanhui Yang. 2026. Clue-RAG: Towards Accurate and Cost-Efficient Graph-based RAG via Multi-Partite Graph and Query-Driven Iterative Retrieval. In *ICDE*.

[47] Chihiro Taguchi, Seiji Maekawa, and Nikita Bhutani. 2025. Efficient Context Selection for Long-Context QA: No Tuning, No Iteration, Just Adaptive-k. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP)*.

[48] Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. 2022. MuSiQue: Multihop Questions via Single Hop Question Composition. *TACL* (2022).

[49] Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. 2023. Interleaving Retrieval with Chain-of-Thought Reasoning for Knowledge-Intensive Multi-Step Questions. In *ACL*. Association for Computational Linguistics, Toronto, Canada, 10014–10037. https://doi.org/10.18653/v1/2023.acl-long.557

[50] Denny Vrandečić and Markus Krötzsch. 2014. Wikidata: A Free Collaborative Knowledgebase. *Commun. ACM* 57, 10 (2014), 78–85.

[51] Jiahao Wang, Weiyu Xie, Mingxing Zhang, Boxin Zhang, Jianwei Dong, Yuening Zhu, Chen Lin, Jingqi Tang, Yaochen Han, Zhiyuan Ai, et al. 2026. From Prefix Cache to Fusion RAG Cache: Accelerating LLM Inference in Retrieval-Augmented Generation. *Proceedings of the ACM on Management of Data* 4, 1 (2026), 1–28.

[52] Bingyang Wu, Yinmin Zhong, Zili Zhang, Shengyu Liu, Fangyue Liu, Yuanhang Sun, Gang Huang, Xuanzhe Liu, and Xin Jin. 2023. Fast Distributed Inference Serving for Large Language Models. *arXiv preprint arXiv:2305.05920* (2023).

[53] Fangyuan Xu, Weijia Shi, and Eunsol Choi. 2024. RECOMP: Improving Retrieval-Augmented LMs with Compression and Selective Augmentation. In *The Twelfth International Conference on Learning Representations (ICLR)*.

[54] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. 2025. Qwen3 technical report. *arXiv preprint arXiv:2505.09388* (2025).

[55] An Yang, Baosong Yang, et al. 2024. Qwen2 Technical Report. *arXiv preprint arXiv:2407.10671* (2024).

[56] Lei Yang, Mohammad Hajiesmaili, Ramesh Sitaraman, and Don Towsley. 2022. Competitive algorithms for online multidimensional knapsack problems. In *Proc. ACM SIGMETRICS*.

[57] Zeyu Yang, Karel Adamek, and Wes Armour. 2023. Part-time Power Measurements: nvidia-smi's Lack of Attention. *arXiv preprint arXiv:2312.02741* (2023).

[58] Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W Cohen, Ruslan Salakhutdinov, and Christopher D Manning. 2018. HotpotQA: A Dataset for Diverse, Explainable Multi-hop Question Answering. In *EMNLP*.

[59] Jiayi Yao, Hanchen Li, Yuhan Liu, Siddhant Ray, Yihua Cheng, Qizheng Zhang, Kuntai Du, Shan Lu, and Junchen Jiang. 2025. Cacheblend: Fast large language model serving for rag with cached knowledge fusion. In *Proceedings of the twentieth European conference on computer systems*. 94–109.

[60] Hao Yuan et al. 2026. DepCache: A KV Cache Management Framework for GraphRAG with Dependency Attention. In *Proceedings of the ACM on Management of Data (SIGMOD)*.

[61] Bowen Zhang and Harold Soh. 2024. Extract, Define, Canonicalize: An LLM-Based Framework for Knowledge Graph Construction. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 9820–9836.

[62] Liangliang Zhang, Zhuorui Jiang, Hongliang Chi, Haoyang Chen, Mohammed ElKoumy, Fali Wang, Qiong Wu, Zhengyi Zhou, Shirui Pan, Suhang Wang, et al. 2025. Diagnosing and Addressing Pitfalls in KG-RAG Datasets: Toward More Reliable Benchmarking. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*.

[63] Nan Zhang, Prafulla Kumar Choubey, Alexander Fabbri, Gabriel Bernadett-Shapiro, Rui Zhang, Prasenjit Mitra, Caiming Xiong, and Chien-Sheng Wu. 2025. SiReRAG: Indexing Similar and Related Information for Multihop Reasoning. In *ICLR*.

[64] Qianchi Zhang, Hainan Zhang, Liang Pang, Yongxin Tong, Hongwei Zheng, and Zhiming Zheng. 2026. Less is More: Compact Clue Selection for Efficient Retrieval-Augmented Generation Reasoning. In *Proceedings of the ACM Web Conference 2026 (WWW)*.

[65] Yanzhao Zhang, Mingxin Li, et al. 2025. Qwen3 Embedding: Advancing Text Embedding and Reranking Through Foundation Models. *arXiv preprint arXiv:2506.05176* (2025).

[66] Lianmin Zheng et al. 2024. SGLang: Efficient Execution of Structured Language Model Programs. In *NeurIPS*.

[67] Yinmin Zhong, Shengyu Liu, Jianbo Chen, et al. 2024. DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving. In *Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*.

[68] Yunhong Zhou, Deeparnab Chakrabarty, and Rajan Lukose. 2008. Budget constrained bidding in keyword auctions and online knapsack problems. In *Proceedings of the 17th international conference on world wide web*. 1243–1244.

Table 7: Summary of Key Notations.

Symbol	Description
	Metric & Problem (Section 2.2)
$\mathcal{S} = \{q_i\}_{i=1}^{n}$	Query batch (n queries)
λ	Request arrival rate
τ_{q_i}	TTFT deadline for query q_i
$\text{Score}(q_i)$	Answer quality $\in [0, 1]$ (e.g., token-level F1)
$\mathcal{J}_\lambda$	QA-Goodput (Eq. 2)
$Q(K_i, q_i)$	Expected answer quality $\mathbb{E}[\text{Score}(q_i) \mid K_i]$
$\mathbf{K} = (K_1, \ldots, K_n)$	Per-query budget vector (decision variable)
π	Serving order / scheduling permutation (decision variable)
	Graph & Retrieval (Phase 1)
$\mathcal{G} = (\mathcal{V}, \mathcal{E}, \mathcal{T}_v, \mathcal{T}_e)$	Global heterogeneous knowledge graph
q, $\mathcal{G}_q$	User query and its induced candidate subgraph
$\mathcal{D}$	Raw document corpus (offline input)
$\mathcal{F}_q$	Topological feature vector of $\mathcal{G}_q$ (Sensor signal)
r_{bridge}	Bridge ratio: fraction of chunks sharing entities with ≥ 2 other chunks
ρ_{overlap}	Entity overlap density: fraction of chunk pairs sharing ≥ 1 entity
n_{entity}	Number of unique entities in $\mathcal{G}_q$
	Adaptive Aggregation (Phase 2)
$S_{\text{diff}}(q)$	Normalized difficulty score $\in [0, 1]$ (Eq. 5)
S_{sys}	Binary backpressure gate: $\mathbb{I}(u(t) > \theta_{\text{bp}})$
θ_{bp}	Backpressure activation threshold
K_q	Final aggregation budget for query q
$K_{\min}$, $K_{\max}$, K_{eff}	Lower / upper / backpressure-adjusted budget bounds
$L_i(K_i)$	Prompt length (tokens) given budget K_i
μ_{p}, μ_{d}	Prefill / decode throughput (tokens / ms)
$p_i = L_i(K_i)/\mu_{\text{p}}$	Estimated prefill time for query q_i
$u(t)$	Instantaneous queue utilization $\in [0, 1]$ (Eq. 6)
	Scheduling (Phase 3)
$U_{\text{slo}}(q)$	SLO urgency = $\min(p_q/\tau_q, 1)$ (Eq. 13)
$\varphi(\cdot)$	Difficulty-aware amplification function
$U^*_{\text{slo}}(q)$	Amplified urgency = $U_{\text{slo}} \cdot \varphi(S_{\text{diff}})$ (Eq. 14)
$C_{\text{prefill}}(q)$	Normalized prefill cost = L_q/L_{ref} (Eq. 15)
$\mathcal{P}(q)$	Composite priority score (Eq. 16)
t_{arr}, $t_{\text{wait}}(q)$	Arrival time and elapsed wait time

A NOTATION TABLE

Table 7 is the full notation table for this paper.

B PROOFS OF THEORETICAL RESULTS

This appendix provides complete proofs for the theoretical results stated in the main text.

B.1 Proof of Theorem 1 (NP-Hardness)

Proof. We reduce from the NP-complete Partition problem [31].

Source problem (Partition). Given a multiset $A = \{a_1, \ldots, a_m\}$ of positive integers with $\sum_{j=1}^{m} a_j = 2B$, decide whether there exists a subset $A' \subseteq A$ such that $\sum_{j \in A'} a_j = B$.

Reduction. Given a Partition instance A, construct a QA-Goodput scheduling instance with query set $\mathcal{S} = \{q_1, \ldots, q_m\}$ and fixed budget vector $\mathbf{K}$:

- Each K_i is chosen such that $L_i(K_i)/\mu = a_i$ (the prefill time of q_i equals a_i);
- $\tau_{q_i} = B$ for all i (common deadline);
- $Q(K_i, q_i) = a_i/(2B) \in (0, 1]$ (quality proportional to processing time);
- $\eta = 1/2$ (threshold).

Correctness. ($\Rightarrow$) Suppose a partition A' exists with $\sum_{j \in A'} a_j = B$. Schedule the corresponding queries first in any order. Their total processing time is $\sum_{i:q_i \in A'} L_i(K_i)/\mu = B \leq \tau_{q_i}$, so all complete by the deadline. The QA-Goodput is $\sum_{i \in A'} Q(K_i, q_i) = B/(2B) = 1/2 \geq \eta$.

($\Leftarrow$) Suppose a serving order π achieves QA-Goodput $\geq \eta = 1/2$. Let S be the set of on-time queries. Then $\sum_{i \in S} Q(K_i, q_i) = \sum_{i \in S} a_i/(2B) \geq 1/2$, i.e., $\sum_{i \in S} a_i \geq B$. Since all on-time queries complete by deadline B, $\sum_{i \in S} L_i(K_i)/\mu = \sum_{i \in S} a_i \leq B$. Combining: $\sum_{i \in S} a_i = B$, so S corresponds to a valid partition.

Since Partition is NP-complete, the restricted decision problem is NP-hard. Since Eq. 3 additionally optimizes over $\mathbf{K}$, it is at least as hard. □

B.2 Proof of Theorem 2 (Joint Dominance)

Proof. Recall that $\mathcal{J}(\mathbf{K}, \pi)$ is the QA-Goodput objective in Eq. (3). Both $(\mathbf{K}^*, \pi^{\text{FCFS}})$ and $(\mathbf{K}^{\text{fix}}, \pi^*)$ are feasible solutions to Eq. (3): $\mathbf{K}^*$ and $\mathbf{K}^{\text{fix}}$ satisfy the budget bounds, while π^{FCFS} and π^* are valid non-preemptive serving orders. Since $(\mathbf{K}^*, \pi^*)$ is globally optimal,

$$\mathcal{J}(\mathbf{K}^*, \pi^*) \geq \mathcal{J}(\mathbf{K}^*, \pi^{\text{FCFS}})$$

and

$$\mathcal{J}(\mathbf{K}^*, \pi^*) \geq \mathcal{J}(\mathbf{K}^{\text{fix}}, \pi^*).$$

Combining the two inequalities yields the result. □

Note. Theorem 2 is a *problem-level* oracle result: it shows that the joint optimum is no worse than the two restricted configurations stated in the theorem. DynaGraph approximates this optimum via greedy priority scheduling and online budget allocation; the effectiveness of this approximation is validated empirically in Section 5.

B.3 Scheduling Approximation under Simplified Model

Theorem 3:*[Scheduling Approximation] Consider a set of queries $\{q_i\}$ with processing times $\{p_i\}$, values $\{Q(K_i, q_i)\}$, and a common deadline τ on a non-preemptive single machine. Taking the better of value-density greedy packing and the best single feasible item achieves $\mathcal{J}_{\text{greedy}} \geq \text{OPT}/2$.* □

Proof. *Step 1 — Knapsack reduction.* With common deadline τ, a subset S of queries is feasibly schedulable if and only if $\sum_{i \in S} p_i \leq \tau$ (non-preemptive, single machine, any ordering within S works as long as the total fits). This is exactly the 0-1 knapsack problem:

$$\max \sum_i v_i x_i \quad \text{s.t.} \quad \sum_i p_i x_i \leq \tau, \quad x_i \in \{0, 1\}.$$

Step 2 — Fractional relaxation. Relaxing x_i to $[0, 1]$, the fractional optimum V_{frac} is obtained by sorting items in decreasing value-density v_i/p_i and greedily filling the knapsack [14]. Since the integer optimum satisfies $\text{OPT} \leq V_{\text{frac}}$, this provides an upper bound.
Step 3 — Greedy gap analysis. Consider the greedy integral packing in the same v_i/p_i order. Let j be the first item that does not fit. The greedy packing value satisfies $V_{\text{pack}} \geq V_{\text{frac}} - v_j$, because the fractional solution assigns j a fractional value of at most v_j. Meanwhile, the maximum single-item value $v_{\max} = \max_{i:p_i \leq \tau} v_i \geq v_j$.
Step 4 — Combine. Taking the better of the greedy packing and the best single item:

$$V_{\text{greedy}} = \max(V_{\text{pack}}, v_{\max}) \geq \max(\text{OPT} - v_j, v_j) \geq \frac{\text{OPT}}{2},$$

where the last inequality holds because for any $a, b \geq 0$ with $a + b \geq \text{OPT}$, $\max(a, b) \geq \text{OPT}/2$. No assumption on the distribution of

processing times $\{p_i\}$ is required. □

Mapping to DynaGraph. In DynaGraph, value $v_i = Q(K_i, q_i)$ is the per-query answer quality, weight $p_i = L_i(K_i)/\mu_p$ is the prefill time (Eq. 4), and the deadline τ corresponds to the SLO target. The theorem is stated under a simplified common-deadline, single-machine surrogate; it provides a design rationale for jointly accounting for query value and processing cost in deadline-constrained scheduling.

B.4 Queue Stability under MMPP Arrivals

Theorem 4: *[Queue Stability] Let arrivals follow an MMPP with peak rate λ_{max} and let μ_p denote the prefill throughput. Under Phase 2's backpressure controller (Eq. 8), the queue process $Q(t)$ is positive recurrent whenever*

$$\lambda_{max} \cdot \frac{K_{min}}{\mu_p} < 1. \tag{17}$$

Moreover, the expected queue length satisfies $\mathbb{E}[Q] \le B/(\epsilon\delta)$, where $\epsilon = \mu_p/K_{min} - \lambda_{max}$, δ is the scheduling window length, and B bounds second-order arrival/service variance. □

Proof. We model the system as a single-server queue with state-dependent service times and Markov-modulated arrivals, using a slotted-time model with slot duration δ.

System model. Let $Q(t)$ be the number of requests in the queue at the beginning of slot t. Arrivals follow an MMPP: a background Markov chain $\{Z(t)\}$ with finite state space $\mathcal{Z}$ modulates the arrival rate, producing $A(t) \sim \text{Poisson}(\lambda_{Z(t)}\delta)$ arrivals per slot. Define $\lambda_{max} = \max_{z\in\mathcal{Z}} \lambda_z$.

The service time for query q in slot t depends on the backpressure state. Under Phase 2's controller (Eq. 8), the per-query budget satisfies $K_q \le K_{eff}(t)$. Under sustained congestion ($u(t) \gg \theta_{bp}$), $K_{eff}(t) \to K_{min}$, so the worst-case service time is bounded by K_{min}/μ_p.

Lyapunov function. Define $V(t) = \frac{1}{2}Q(t)^2$. The one-slot conditional drift is:

$$\Delta V(t) \triangleq \mathbb{E}\big[V(t+1) - V(t) \mid Q(t), Z(t)\big]. \tag{18}$$

Using the dynamics $Q(t+1) = \max(Q(t) - D(t), 0) + A(t)$:

$$\Delta V(t) \le Q(t) \cdot \big(\mathbb{E}[A(t) \mid Z(t)] - \mathbb{E}[D(t) \mid Q(t)]\big) + B, \tag{19}$$

where $B = \frac{1}{2}\big((\lambda_{max}\delta)^2 + (\mu_p\delta/K_{min})^2\big)$ bounds the second-order terms.

Service rate under backpressure. When $Q(t)$ is large (more precisely, when $u(t) > \theta_{bp}$), Phase 2 drives $K_{eff}(t) \to K_{min}$, so every query's service time is bounded by K_{min}/μ_p. The effective service rate per slot is at least $\mu_p\delta/K_{min}$ queries.

Negative drift. Substituting into Eq. 19:

$$\Delta V(t) \le Q(t) \cdot \delta\left(\lambda_{max} - \frac{\mu_p}{K_{min}}\right) + B. \tag{20}$$

When $\lambda_{max} \cdot K_{min}/\mu_p < 1$ (Eq. 17), define:

$$\epsilon \triangleq \frac{\mu_p}{K_{min}} - \lambda_{max} > 0.$$

Then $\Delta V(t) \le -\epsilon\delta \cdot Q(t) + B$, which is negative for $Q(t) > B/(\epsilon\delta)$.

Foster–Lyapunov criterion. By the Foster–Lyapunov theorem [38], the existence of a Lyapunov function with negative drift outside a finite set $\{Q : Q \le B/(\epsilon\delta)\}$ implies positive recurrence of the joint process $(Q(t), Z(t))$. Since $\{Z(t)\}$ is ergodic by assumption and the drift condition holds uniformly over all states $z \in \mathcal{Z}$ (because we use peak rate λ_{max}), $Q(t)$ is positive recurrent, i.e., $\mathbb{E}[Q] < \infty$.

Quantitative queue-length bound. By the quantitative Foster–Lyapunov theorem [39], taking expectations over the stationary distribution:

$$\mathbb{E}[Q] \le \frac{B}{\epsilon\delta}. \tag{21}$$

This bound is used in the starvation analysis (Appendix B.5). □

Mapping to DynaGraph. Eq. 17 yields a closed-form deployment rule: the system is stable whenever $K_{min} < \mu_p/\lambda_{max}$. This condition is verified once at deployment time using the profiled prefill throughput μ_p and the expected peak arrival rate λ_{max}. The key mechanism is Phase 2's backpressure (Eq. 8): under sustained congestion, $K_{eff}(t) \to K_{min}$ drives all service times to their minimum, ensuring the queue drains faster than it fills. Without backpressure, a fixed budget K_{max} would require the much stronger condition $\lambda_{max} \cdot K_{max}/\mu_p < 1$, which may be violated under bursty workloads.

B.5 Aging under Stable Fixed-SLO Workloads

Proposition 5 (Finite Expected Wait with Aging). *Assume a fixed SLO tier $\tau_q = \tau$, bounded base priorities $\mathcal{P}(q) \in [\mathcal{P}_{min}, \mathcal{P}_{max}]$, bounded service time s_{max}, work-conserving service with arrival-order tie breaking, and the stability condition of Theorem 4. Under*

$$\mathcal{P}_{aged}(q,t) = \mathcal{P}(q) + \frac{\Delta_{age}}{\tau}\max(t_{wait}(q) - T_{age}, 0), \tag{22}$$

a tagged query arriving in stationarity has finite expected wait. In particular,

$$\mathbb{E}[W_q] \le T_{age} + \frac{\tau}{\Delta_{age}}(\mathcal{P}_{max} - \mathcal{P}(q))_+ + s_{max}(\mathbb{E}[Q] + 1).$$

Proof. After T_{age}, the tagged query's priority grows at the common rate Δ_{age}/τ. It reaches the largest possible base priority after at most the second term in the bound. Later arrivals have base priority at most $\mathcal{P}_{max}$ and the same aging slope, so they cannot overtake the tagged query once this crossing occurs; arrival-order tie breaking resolves equality. The tagged query may still wait for requests already ahead of it. Their remaining service is bounded by $s_{max}(Q+1)$. Taking expectations under the stable stationary queue gives the stated finite bound. □

Heterogeneous deadlines. For mixed SLO tiers, the same argument requires either a deployment-chosen uniform aging slope or a common positive lower bound across all aging slopes. Without this condition, we retain aging only as an implementation safeguard and do not claim any deterministic per-query multiple of τ_q.

B.6 Design Rationale for Exponential Backpressure

The following classical result from online knapsack theory motivates DynaGraph's choice of exponential decay for the budget ceiling $K_{\text{eff}}(t)$ (Eq. 8).

Theorem 6:*[Optimal Reservation for Online Knapsack [68]] Consider a stream of items arriving online, each with value-to-weight ratio $v \in [L, U]$, and a knapsack of capacity C.* **(a)** *No deterministic online algorithm achieves a competitive ratio better than* $\ln(U/L) + 1$ [68]. **(b)** *The exponential threshold $\Psi(z) = L \cdot (U/L)^z$, which admits an item iff $v \geq \Psi(z)$ at utilization $z \in [0, 1]$, achieves this bound and is therefore optimal* [10]. **(c)** *A linear threshold achieves only $O(\sqrt{U/L})$, which is strictly worse when $U/L > e$* [56]. □

Proof. **Part (a): Lower bound.** Consider a single knapsack of capacity C and items with value-to-cost ratio in $[L, U]$. An adversary constructs two input sequences: σ_1 consists of items with ratio L filling the knapsack, followed by items with ratio U; σ_2 consists only of items with ratio L. Any deterministic algorithm that reserves capacity for high-value items (performing well on σ_1) loses on σ_2, and vice versa. Balancing across all such adversarial constructions yields the lower bound $\ln(U/L) + 1$ [68].

Part (b): Optimality of the exponential threshold. Define the threshold function $\Psi : [0, 1] \to [L, U]$ as:

$$\Psi(z) = L \cdot \left(\frac{U}{L}\right)^z, \tag{23}$$

where $z \in [0, 1]$ is the fraction of capacity consumed. The algorithm admits item i with ratio v_i if and only if $v_i \geq \Psi(z)$.

We verify the competitive ratio via the primal-dual method of Chakrabarty et al. [10]. Assign a dual variable $\psi(z) = \Psi(z)$ to the capacity constraint. For any admitted item i at utilization z_i, the primal gain is $v_i \geq \Psi(z_i)$. Integrating the dual over the utilization trajectory:

$$\int_0^Z \Psi(z)\, dz = \int_0^Z L \cdot \left(\frac{U}{L}\right)^z dz = \frac{L}{\ln(U/L)} \left[\left(\frac{U}{L}\right)^Z - 1\right],$$

where Z is the final utilization. The ratio of primal value to dual cost is bounded by $\ln(U/L) + 1$ for all input sequences, matching the lower bound from part (a).

Part (c): Sub-optimality of the linear threshold. A linear threshold $\Psi_{\text{lin}}(z) = L + (U - L) \cdot z$ admits items with ratio in $[L, L + (U - L)z]$ at utilization z. Under an adversarial arrival sequence that concentrates high-value items ($v_i \approx U$) at $z \to 1$, the algorithm has already consumed capacity on medium-value items, yielding a competitive gap of $\Theta(\sqrt{U/L})$. Yang et al. [56] provide the tight analysis for multidimensional knapsack, which specializes to the single-dimension case here. □

Mapping to DynaGraph. In DynaGraph, each query q corresponds to an "item" with value $Q(K_q, q)$ (answer quality) and cost proportional to K_q (prefill tokens). The ratio range $[L, U]$ maps to $[K_{\min}, K_{\max}]$. DynaGraph's effective ceiling (Eq. 8) instantiates Ψ in budget space:

$$K_{\text{eff}}(u) = K_{\min} + (K_{\max} - K_{\min}) \cdot \exp\left(-\ln \frac{K_{\max}}{K_{\min}} \cdot \frac{u - \theta_{\text{bp}}}{1 - \theta_{\text{bp}}}\right),$$

which is monotonically decreasing in u and bounded in $[K_{\min}, K_{\max}]$. The shift by θ_{bp} defines a free zone ($u \leq \theta_{\text{bp}}$) where capacity is unconstrained, following the slack extension of Buchbinder and Naor [9]. This result *motivates* the choice of exponential decay over linear alternatives (validated empirically in Section 5.4, Figure 7(a)), rather than serving as a direct end-to-end guarantee for the full online problem.

Compatibility with Queue Stability. The queue stability guarantee (Theorem 4) uses $K_{\min}$ as the worst-case budget under saturation. Since $K_{\text{eff}}(t)$ is monotonically decreasing and $K_{\text{eff}}(t) \to K_{\min}$ as $u(t) \to 1$, the Lyapunov argument applies directly with $K_{\min}$ as the service-time bound.

C DETAILED EXPERIMENT SETTING

This appendix provides the hardware and software stack, workload protocol, dataset and model details, and DynaGraph's configuration defaults used throughout the evaluation. Baseline-specific invocation details and vLLM launch commands are available in the code repository [4].

C.1 Hardware and Software Stack

Table 8: Hardware and software environment.

Component	Specification
GPU	2× NVIDIA RTX 4090 (24 GB each)
Host memory	128 GB DDR5
CUDA	12.8
Python	3.11
PyTorch	2.10.0
LLM engine	vLLM 0.17.1 (main) / SGLang ≥0.5.9
Embedding index	FAISS (CPU) 1.13.2

All tensor-parallel-2 runs pin both GPUs. We use the FLASHINFER attention backend for Qwen3.5 and Qwen3 models with FP8 KV-cache (–kv-cache-dtype fp8). Priority scheduling is enabled via –scheduling-policy priority (vLLM) or –enable-priority-scheduling (SGLang).

C.2 Workload Protocol

Unless otherwise stated for a specific RQ, all experiments use the following online serving protocol.

Table 9: Default workload configuration.

Parameter	Value
Arrival process	Zipf popularity, $\alpha = 1.0$
Unique queries $\lvert\mathcal{S}\rvert$	1 000
Total requests N	3 000
Request rate λ	30 qps
Client concurrency	64
TTFT deadlines τ	{200, 500, 2 000} ms
Warm-up (discarded)	300 requests
DynaGraph $K_{\min}$ / $K_{\max}$	10 / 20
DynaGraph θ_{bp}	0.7
DynaGraph decay shape	Exponential (Eq. 8)

Table 13: Phase 3 scheduler defaults.

Parameter	Description	Default
Strategy	Priority computation	Composite
w_{slo}	Weight on amplified urgency U^*_{slo}	0.6
w_{cost}	Weight on prefill cost C_{prefill}	0.4
Difficulty amplification	Boost U_{slo} when $S_{\text{diff}} > 0.5$	Enabled
μ_{p}	Calibrated prefill throughput	Per-model profiled

Table 14: Per-RQ experiment configuration summary.

RQ	LLM	TP	Datasets	Compared against
RQ1	Qwen3.5-4B	2	2Wiki, HotpotQA, MuSiQue	SiReRAG, KET-RAG, ClueRAG, HippoRAG
RQ2	Qwen3-4B, Qwen2.5-7B	2	2Wiki	CacheBlend, DepCache, VanillaGRAG
RQ3	Qwen3.5-4B	2	2Wiki	Decay / feature ablations
RQ4	Qwen3-4B	1	2Wiki	Static baseline (K=20+FCFS)
RQ5	Qwen3.5-{0.8–35B}	2	2Wiki (+full corpus)	Per-model capacity sweep
RQ6†	Qwen3.5-4B	2	2Wiki, HotpotQA	Adaptive-K, RAG-CSM, CompSelect
RQ7†	Qwen3-4B	1	2Wiki	FCFS, SJF, EDF, Phase ablations

†Appendix experiments (Section E).

Each run consists of a 300-request warm-up (discarded) followed by the 3 000-request measured segment. Metrics are computed on the measured segment only. Per-RQ deviations: RQ4 varies $K_{\min} \in [3, 15]$, $\theta_{\text{bp}} \in [0.1, 1.0]$, and $\lambda \in [2, 30]$; RQ5 varies the LLM backbone and operating λ.

C.3 Datasets

Table 10: Dataset statistics.

Dataset	Full queries	Full corpus	Sampled	Used in
2WikiMultihopQA	12 576	56 687	6 119	RQ1–RQ7
HotpotQA	7 405	66 635	9 811	RQ1, RQ3–RQ7
MuSiQue	2 417	11 656	11 656	RQ1, RQ5

Following the HippoRAG protocol [19], we randomly sample 1 000 queries per dataset with their supporting and distractor passages as the retrieval corpus $\mathcal{D}$ (≈6–12 K passages per dataset). For RQ5 corpus scalability on 2Wiki, we additionally use the full development set (12 576 queries; ≈57 K passages).

C.4 Models

All model weights are downloaded from the Hugging Face Hub.

Table 11: Models used in evaluation.

Role	Model	Used in
Default LLM (hybrid Mamba)	Qwen3.5-4B	RQ1, RQ3, RQ5, RQ6
Pure-attention LLM	Qwen3-4B-Instruct-2507	RQ2, RQ4, RQ7
Medium LLM	Qwen2.5-7B-Instruct	RQ2
Cross-scale LLMs	Qwen3.5-{0.8B, 2B, 9B, 35B-A3B}	RQ5
Embedding	Qwen3-Embedding-0.6B	Phase 1 retrieval
Reranker	BGE-Reranker-v2-M3	Phase 1 rerank

C.5 DynaGraph Configuration

Tables 12 and 13 list the default configuration for DynaGraph's Phase 2 controller and Phase 3 scheduler. All ablation experiments (RQ3, RQ4, RQ7) vary one parameter at a time while keeping the rest at these defaults.

Table 12: Phase 2 controller defaults.

Parameter	Description	Default
$K_{\min}$, $K_{\max}$	Budget bounds	10, 20
θ_{bp}	Backpressure threshold	0.7
M	Hop-1 semantic-anchor depth	5
Decay shape	$K_{\text{eff}}(t)$ under congestion	Exponential
K mapping	$S_{\text{diff}} \to K_q$	Linear
$\mathbf{w}$	Weights of $(r_{\text{bridge}}, \rho_{\text{overlap}}, n_{\text{entity}})$	(0.4, 0.3, 0.3)

C.6 Per-RQ Experiment Matrix

Table 14 provides a compact overview of each RQ's LLM, tensor parallelism, datasets, and comparison targets.

D BASELINE CONFIGURATION DETAILS

All external baselines use their officially released code and reported settings where available, with serving adaptations explicitly documented. Within each comparison, systems share the stated underlying LLM, hardware, workload generator, and evaluated request set (Section 5.1). Method-required serving backends and whether retrieval, routing, or selection costs are included in TTFT are specified below. We report the key configurations and adaptation rationales for each baseline.

D.1 GraphRAG Baselines (Table 3)

For DynaGraph and the four GraphRAG baselines, retrieval and context construction are precomputed before traffic replay; the reported TTFT measures generation-serving latency.

SiReRAG [63]. We use the official release with `tree_depth=1`, `similarity_top_k=2`, and BGE embedding. This is a best-effort adaptation of the released pipeline to our corpus and serving stack: the released default `tree_depth=3` increases indexing cost without changing the retrieved top-20 pool on our corpora. The top-20 retrieved contexts per query form the candidate pool, consistent with the context budget used for the other baselines.

KET-RAG [22]. We use the official release with the `keyword` strategy (θ=0.5), a maximum context length of 12 000 tokens, 10 top mapped entities, and 10 top relationships, following the configuration in the official README.

ClueRAG [46]. We use the official release with cosine selection (α=1.0), BGE-M3 embedding and reranking, top-K=3 entities, top-M=5 candidate chunks, and traversal depth 3. Entity linking, knowledge anchoring, and dynamic query updating are enabled, following the official defaults.

HippoRAG [19]. We follow the official reproduction configuration: retrieval top-K=200, linking top-K=5, at most three QA steps, synonym-edge top-K=2047, similarity threshold 0.8, and damping factor 0.5. The main experiment uses QA top-K=10, tuned for our serving corpus. Under the official default QA top-K=5, the same 2Wiki workload yields F_1=0.410 with p50 TTFT of 355 ms. Both settings lead to the same qualitative conclusions.

Table 15: Phase-1 candidate-source ablation (2Wiki, Qwen3.5-4B, Zipf@30 QPS).

Phase-1 source	Pool	avg K	Toks	F1	p95 (ms)	$\mathcal{J}$@500
C_q^{chunk} only	30.0	10.08	1825	.499	370	61.2%
C_q^{graph} only	913.6	9.91	1905	.241	337	17.0%
$C_q^{\text{chunk}} \cup C_q^{\text{doc}}$	52.9	10.08	2598	.510	581	59.3%
full union (DynaGraph)	63.2	10.08	**2391**	**.773**	473	**79.4%**

D.2 KV-Cache Serving Baselines (Table 4)

CacheBlend [59]. We integrate the official release through LMCache,[2] because the official vllm_blend backend remains under active development. We use segment-level KV offloading with a CPU cache and align chunk separators with context boundaries. CacheBlend is evaluated on Qwen3-4B.

DepCache [60]. We use the official SGLang-based release with DepAttn dependency-aware attention. Document text and KG triples are converted into the DepPrompt format. DepCache is evaluated on Qwen3-4B [54] and Qwen2.5-7B [55]. The Qwen3-4B latency-derived entries in Table 4 originate from a concurrency-2 stdout diagnostic, whereas the matched serving protocol for the other rows uses concurrency 64. We therefore retain these entries only as diagnostic evidence and exclude them from matched-protocol latency and QA-Goodput rankings.

D.3 Context-Selection Baselines (Table 16)

All context-selection baselines receive the same Phase 1 candidate pool from DynaGraph and differ only in how they select the final evidence subset. In Table 16, selector outputs are precomputed and selector inference is excluded from TTFT. Appendix E.7 separately reports a live RAG − CSM comparison in which selector inference is timed for every request and its unrounded score enters the same Phase 3 priority path.

Adaptive-K [47]. We use a score-ratio budget heuristic following the released Adaptive-k interface, with ignore_extreme_tail=0.1 and retrieve_more=5, applied to the Phase 1 candidate pool. Our adaptation uses a score-ratio cutoff rather than the official largest_gap rule because the latter assumes access to raw retriever similarity scores, which our unified reranked pool replaces. We therefore label this configuration as a heuristic adaptation.

RAG-CSM [15]. We use the official 2WikiMultihopQA and HotpotQA CSM checkpoints. Selection is applied to the top-10 retrieved contexts with threshold 0; contexts with CSM scores above 0 are retained. We enforce min_docs=1 to retain at least one context. For the live audit, we verify the checkpoint identity and reproduce the released scores; selector inference is timed for every request, and its unrounded score enters the same Phase 3 priority path.

CompSelect [64]. We apply the extractor stage from the official release to the retrieved pool. The reranker and adaptive-truncator stages are disabled in this best-effort serving adaptation because they operate on retriever-native scores, which our unified reranked pool replaces.

D.4 AdaptiveRAG [26]

We use the official Adaptive-RAG code and evaluation pipeline and train a T5-large router under the prescribed strict out-of-distribution split: 3 160 training and 1 170 validation examples, with no 2WikiMultihopQA overlap. The router selects among zero-step, single-step (BM25, top-15), and multi-step (IRCoT, BM25, top-6 per step) execution using the released prompts and demonstrations. We adapt the pipeline to the paper corpus and Qwen3.5-4B generation endpoint. All route-induced LLM calls are retained, and isolated router latency is included in TTFT. The router checkpoint is trained under the released protocol rather than taken from an upstream official checkpoint.

[2]We minimally modified the CacheBlend codebase and its text-segmentation logic to support the Qwen3 model series.

D.5 Internal Controls

VanillaGRAG. This control uses DynaGraph's Phase 1 retrieval with fixed K=20, no Phase 2 or Phase 3 optimization, and FCFS scheduling. It serves as the full-context quality reference and the unoptimized latency reference.

Full K=20 (C1). All 20 Phase 1 candidates are retained and served using FCFS. This is an internal static-budget control.

Uniform K=10. A fixed budget of 10 candidates is used for every query with FCFS scheduling. This is an internal equal-budget control.

Scheduling ablations (FCFS/SJF/EDF, C2/C3). These internal ablations vary the scheduling policy and adaptive aggregation independently. They are design-choice controls rather than external baselines.

E ADDITIONAL EXPERIMENT RESULTS

This appendix provides supplementary experiments that extend the main evaluation (Section 5). RQ6 (Section E.2) compares DynaGraph's context selection against training-free and trained baselines under the same precomputed retrieval setting as RQ1–RQ4. We further include an evidence-omission diagnostic (Section E.3) to show how missing gold evidence can elongate generation and how DynaGraph limits its propagation into queueing and strict-SLO failures. RQ7 (Section E.4) provides detailed phase ablations (Phase 2 vs. Phase 3) and scheduling-strategy comparisons (FCFS, SJF, EDF, composite) on two different LLM architectures, supplying the fine-grained evidence behind the joint design claims in Section 3.2 and Theorem 2.

E.1 Phase-1 Candidate-Source Ablation

Effect of Phase-1 candidate sources. We evaluate the contribution of different Phase-1 candidate sources independently of downstream serving decisions. We fix the serving layer, budget policy, reranker, and reader, hold the realized budget at avg $K \approx 10$, and vary only the candidate source. As shown in Table 15, adding parent documents marginally improves F1 from 0.499 to 0.510 but increases prompt length by 42%, whereas the full union achieves 0.773 F1 and 79.4% $\mathcal{J}$@500; graph retrieval alone achieves only 0.241 F1. This

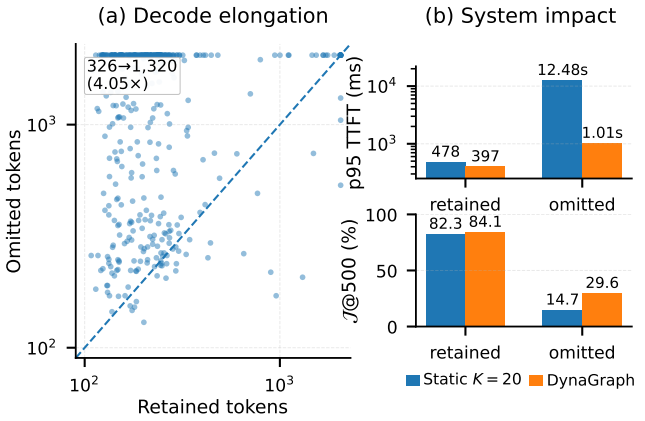


Figure 10: Evidence omission and decode propagation. (a) Replacing one gold-support document with length-matched non-gold evidence elongates generation. (b) Under queue replay, DynaGraph limits the propagation of omission-induced long decodes into p95 TTFT and $\mathcal{J}$@500 degradation compared with static K=20.

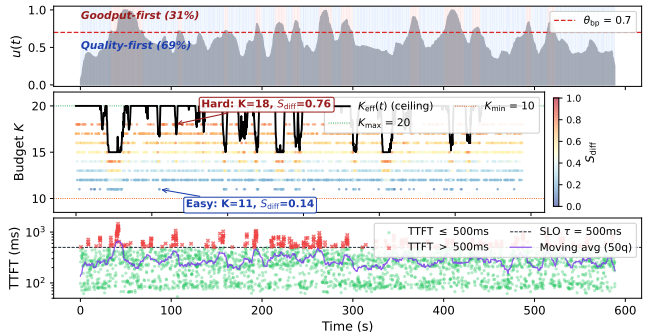


Figure 9: Closed-loop budget allocation in action. Top: queue utilization $u(t)$ alternates between quality-first and goodput-first regimes. Middle: $K_{\text{eff}}(t)$ decays under congestion while per-query K_q preserves difficulty differentiation (darker = harder). Bottom: resulting per-query TTFT.

Table 16: Context aggregation comparison (Qwen3.5-4B TP2, Zipf@30 QPS, 1000q×3000). Baselines use FCFS scheduling; DynaGraph uses joint Phase 2+3.

Method	F1↑	avg K	p50 (ms)↓	p99 (ms)↓	SLO@500↑	$\mathcal{J}$@200↑	$\mathcal{J}$@500↑	$\mathcal{J}$@2s↑
2WikiMultiHopQA								
Full K=20 (C1)	.784	20.0	305	9127	83.5%	24.7%	52.5%	61.5%
Uniform K=10	.780	10.0	223	1740	94.1%	26.3%	57.5%	60.6%
Adaptive-K [47]	.788	11.2	229	3220	94.1%	24.2%	57.0%	59.6%
RAG-CSM [15]	.522	6.6	162	2385	97.7%	28.7%	46.8%	47.3%
CompSelect [64]	.328	2.7	107	1155	98.2%	28.1%	31.0%	31.4%
DynaGraph	.781	10.1	215	15097	98.6%	**41.5%**	**81.8%**	**82.8%**
HotpotQA								
Full K=20 (C1)	.776	20.0	220	2032	95.6%	28.6%	68.1%	70.7%
Uniform K=10	.767	10.0	220	2034	94.8%	28.5%	66.8%	69.9%
Adaptive-K [47]	.767	10.0	220	2036	96.0%	27.7%	67.4%	69.5%
RAG-CSM [15]	.555	6.4	158	2005	97.8%	35.1%	47.2%	47.8%
CompSelect [64]	.602	5.0	115	5090	97.9%	46.2%	51.1%	51.1%
DynaGraph	.765	10.1	220	2782	95.9%	29.9%	**68.3%**	**70.3%**

controlled comparison isolates Phase-1 retrieval from the serving layer and shows that combining complementary candidate sources is critical to DynaGraph's quality–latency gains.

E.2 RQ6: Context Aggregation Comparison

RQ6 compares DynaGraph's context selection against four baselines spanning three design families: static budgets, training-free score thresholding, and trained selectors. All methods use the same Phase 1 retrieval output. The baselines use FCFS scheduling, whereas DynaGraph retains its joint Phase 2–3 strategy. Experiments are conducted on 2Wiki and HotpotQA with Qwen3.5-4B at Zipf@30 QPS (Table 16).

DynaGraph vs training-free baselines. On 2Wiki, DynaGraph achieves $\mathcal{J}$@500 of 81.8%, outperforming the strongest training-free baseline (Uniform K=10: 57.5%) by +24.3 pp, while using a comparable average budget (avg K: 10.1 vs 10.0) and maintaining similar F1 (0.781 vs 0.780). Adaptive – K [47] thresholds on reranker score ratios, producing avg K=11.2, yet achieves only 57.0% $\mathcal{J}$@500—on par with Uniform K=10. This confirms that context selection quality alone (even with near-perfect offline NDCG of 0.974) does not translate to online goodput without serving-aware scheduling. On HotpotQA, all training-free methods cluster within 66.8–68.3% $\mathcal{J}$@500, with DynaGraph marginally ahead (68.3%); this narrower gap reflects HotpotQA's lower multi-hop complexity, where most queries are answerable with moderate context and scheduling provides less leverage.

Trained context selectors. RAG – CSM [15] trains a BERT-based scorer on per-document context influence, compressing avg K to 6.6 (−67% vs K=20) and achieving the second-lowest p50 (162 ms). However, F1 drops to 0.522 (−26 pp vs DynaGraph), indicating that the learned scorer over-prunes bridging evidence required for multi-hop reasoning. CompSelect [64] applies LoRA-tuned sentence extraction, reducing avg K to 2.7 (−86%) with the lowest p50 (107 ms), but F1 collapses to 0.328 (−45 pp), yielding $\mathcal{J}$@500 of only 31.0%. These results reveal a *K-cliff* phenomenon: reducing avg K from 10 to 6 incurs a 26 pp F1 penalty, and from 10 to 3 a 45 pp penalty, whereas reducing from 20 to 10 costs only 0.4 pp. Gold answer documents are retained in the top-K, but the bridging context required for multi-hop chains falls outside the aggressively compressed budget.

Retrieval-serving gap. Adaptive – K achieves the highest offline retrieval quality (NDCG@10: 0.974 on 2Wiki), yet delivers identical $\mathcal{J}$@500 as Uniform K=10 (57.0% vs 57.5%). DynaGraph's NDCG@10 is lower (0.941), but its $\mathcal{J}$@500 is 81.8%—a +24.8 pp gap. This *retrieval-serving gap* demonstrates that offline ranking metrics cannot predict online goodput; the critical differentiator is queue-aware scheduling (Phase 3), which converts the per-query cost variance created by difficulty-aware K allocation (Phase 2) into system-level goodput gains.

Complementarity with trained methods. Both RAG – CSM and CompSelect are orthogonal to DynaGraph's training-free pipeline: their learned scoring functions could serve as drop-in replacements for the reranker in Phase 1, potentially improving the quality of the candidate set C_q that Phase 2 subsequently compresses. However, their current implementations assume static K and FCFS scheduling, and integrating them with load-aware backpressure and priority scheduling remains unexplored. The results suggest that the largest gains lie not in better context selection per se, but in coupling any selection method with serving-aware execution.

Table 18: Scheduling strategy comparison (2Wiki, Qwen3-4B TP1, Zipf@30 QPS, 1000q×3000 requests). Each method uses its natural K strategy.

Method	K strat.	Sched	F1↑	avg K	p50↓	p95↓	p99↓	$\mathcal{J}$@200↑	$\mathcal{J}$@500↑
C1 (Static)	Fixed 20	FCFS	.759	20.0	359	1560	4122	19.0%	54.3%
SJF K=20	Fixed 20	SJF	.756	20.0	344	2022	4228	21.3%	53.7%
EDF K=20	Fixed 20	EDF	.756	20.0	329	1034	5469	20.8%	60.3%
SJF K=10	Fixed 10	SJF	.742	10.0	133	343	1465	61.5%	78.0%
C3 (AdpK only)	Diff, no BP	FCFS	.750	14.7	181	501	3000	45.5%	76.5%
AdpK+BP	Diff+BP	FCFS	.755	12.5	173	476	2969	47.8%	77.4%
DynaGraph	Diff+BP	Comp.	.755	12.5	**143**	**427**	3310	**55.6%**	**77.9%**

Table 17: Phase ablation (Qwen3.5-4B, 2Wiki, Zipf@30 QPS). Configurations follow Table 1; C4 automates expert labels via Phase 1 topological features with backpressure feedback.

Config	K	Sched	F1	Acc	p50 (ms)	SLO@500	$\mathcal{J}$@500
C1 (Static)	Fixed 20	FCFS	**.800**	**.857**	5892	0.0%	0.0%
C2 (P3 only)	Fixed 20	Composite	.793	.859	407	57.8%	41.6%
C3 (P2 only)	Adaptive	FCFS	.787	.844	4076	0.0%	0.0%
C4 (DynaGraph)	Joint	+BP	.787	.841	**220**	**94.9%**	**72.8%**

E.3 Evidence Omission and Decode Propagation

Evidence omission and decode propagation. To test whether missing evidence can turn a retrieval failure into a serving bottleneck, we combine a controlled per-query intervention with a queue-level replay. We select 400 real 2Wiki bridge/comparison questions with multiple annotated gold-support documents and, at fixed K=20, replace exactly one gold support with length-matched non-gold evidence, keeping prompt length nearly unchanged (p95 relative difference: 1.21%). This omission increases mean completion length from 326 to 1,320 tokens, raises max-token truncation from 3.25% to 46.25%, and reduces F1 from 0.810 to 0.325 (Figure 10(a)). We then inject the same omission before Phase 2 and replay the workload under static K=20 and DynaGraph (Figure 10(b)). The omission-induced p95 TTFT increase drops from 12.00 s under static K=20 to 0.62 s under DynaGraph; the SLO@500 loss drops from 47.92 to 6.56 percentage points, and the $\mathcal{J}$@500 degradation is reduced by 13.15 points. The F1 interaction is not significant, indicating that DynaGraph does not recover the missing semantics, but limits the propagation of omission-induced long decodes into queueing and strict-SLO failures.

E.4 RQ7: Phase Ablation and Scheduling Strategy Analysis

The main text (RQ2) establishes that DynaGraph's closed-loop interaction between Phase 2 (adaptive budgeting) and Phase 3 (SLO-aware scheduling) is necessary for high QA-Goodput. RQ7 provides detailed ablations supporting this claim: (A) a phase-level ablation isolating Phase 2 and Phase 3 on Qwen3.5-4B (Table 17), and (B) a scheduling strategy comparison decomposing each component's contribution on Qwen3-4B (Table 18).

(A) Phase ablation (Table 17). We ablate Phase 2 and Phase 3 on Qwen3.5-4B (hybrid Mamba, TP=2) using four configurations that follow the progressive elimination structure in Table 1: C1 (Fixed K=20 + FCFS), C2 (Fixed K=20 + composite scheduling), C3 (adaptive K + FCFS), and C4 (DynaGraph: adaptive K + composite scheduling + backpressure). Neither KV-cache baseline (CacheBlend, DepCache) can run on Qwen3.5-4B's Mamba layers, making this ablation the only way to evaluate serving strategies on hybrid architectures.

(A1) Phase 2 alone reduces cost but cannot meet SLOs. C3 applies difficulty-aware K allocation under FCFS, reducing avg K from 20 to 14.7 and lowering mean prefill cost by 27%. However, SLO@500 remains 0% (p50: 4,076 ms) because FCFS allows a small number of heavy queries—those with both high prefill and long decode—to block the queue. Adaptive budgeting creates per-query cost heterogeneity, but without a scheduler to exploit it, head-of-line blocking persists.

(A2) Phase 3 alone improves SLO but wastes budget. C2 applies composite priority scheduling with uniform K=20, lifting SLO@500 to 57.8% and p50 from 5,892 ms to 407 ms. However, with every query allocated the same budget, there is no cost variance to exploit: the scheduler can reorder but cannot reduce aggregate prefill load, capping $\mathcal{J}$@500 at 41.6%. This confirms the insight from Section 3.2: scheduling uniformly heavy requests merely redistributes latency without reducing it.

(A3) Joint Phase 2+3 resolves both bottlenecks. C4 (DynaGraph) combines adaptive K with composite scheduling and backpressure. Adaptive K compresses easy queries while preserving larger budgets for structurally complex ones, creating per-query cost variance; the scheduler interleaves short and long prefills to minimize queue waiting time; under congestion, backpressure further compresses K_{eff} (Eq. 8). The result: SLO@500 reaches 94.9% and $\mathcal{J}$@500 reaches 72.8%, with F1 only −0.013 below C1. This empirically confirms the benefit of combining Phases 2 and 3 under this setting.

(A4) Architecture generality. Because the optimization operates above the inference engine—controlling prompt length and request ordering rather than modifying attention computation—DynaGraph extends to hybrid architectures such as Qwen3.5-4B's Mamba layers. KV-cache methods (CacheBlend, DepCache) depend on specific attention layouts and cannot run on this architecture, as shown in RQ2.

(B) Scheduling strategy comparison (Table 18).

We compare scheduling strategies on Qwen3-4B (TP1, higher per-query load pressure) to quantify each component's marginal contribution. Each method uses its natural K allocation to reflect realistic deployment. The table decomposes the path from C1 (Static) to DynaGraph into four incremental steps.

(B1) Scheduling alone has limited effect with uniform K. With fixed K=20, replacing FCFS with SJF improves $\mathcal{J}$@200 by only +2.3 pp (21.3% vs 19.0%), and EDF by +1.8 pp. When all prompts are uniformly heavy, the scheduler can reorder requests but cannot reduce total prefill load. SJF slightly improves median latency (344 vs 359 ms) but worsens p95 (2,022 vs 1,560 ms), because prioritizing short jobs starves long ones. EDF improves p95 (1,034 ms) by respecting deadlines but at the cost of p99 (5,469 ms), as late-arriving hard queries miss their window. Neither strategy achieves meaningful $\mathcal{J}$@500 improvement (53.7–60.3% vs C1's 54.3%). This validates Obs. 3 (Section 3.2): scheduling uniformly heavy requests merely redistributes latency across the queue.

(B2) Adaptive K is the dominant lever. C3 (difficulty-aware K, no backpressure, FCFS) lifts $\mathcal{J}$@200 from 19.0% to 45.5% (+26.5 pp) and $\mathcal{J}$@500 from 54.3% to 76.5% (+22.2 pp), dwarfing any scheduling-only improvement. The gain comes from two sources: easy queries

Table 19: End-to-end live deployment. Retrieval and generation run concurrently. Qwen3.5-4B and Qwen3-4B, λ=5 qps, 1 000 queries.

LLM	Dataset	Retr (ms)	Gen (ms)	E2E p50	E2E p95	F1	Acc
Qwen3.5-4B	2Wiki	178	414	613	1 157	.797	.842
	HotpotQA	270	231	529	724	.714	.670
	MuSiQue	300	339	679	922	.399	.421
Qwen3-4B	2Wiki	180	304	502	888	.764	.803
	HotpotQA	254	138	395	525	.698	.650
	MuSiQue	352	276	614	836	.371	.341

Table 20: E2E scalability: DynaGraph vs ClueRAG on HotpotQA with increasing corpus size. Ratio = ClueRAG / DynaGraph E2E p50.

Scale	Queries	Corpus	DynaGraph p50 (ms)	ClueRAG p50 (ms)	Ratio
1×	1 000	∼9.8 K	360	609	1.7×
3×	3 000	∼28 K	369	963	2.6×
6×	6 000	∼55 K	386	1 614	4.2×
Full	7 405	∼67 K	392	2 076	**5.3**×

Table 21: Cross-model generalization (2WikiMultihopQA, DynaGraph, 1000 unique queries, 3000 Zipf requests, 2×RTX 4090).

Model	λ	F1↑	Acc↑	p50↓ (ms)	p95↓ (ms)	SLO@500↑	$\mathcal{J}$@200↑	$\mathcal{J}$@500↑	$\mathcal{J}$@1s↑
Qwen3.5-0.8B	200	.552	.587	84	155	100.0%	49.0%	49.4%	49.4%
Qwen3.5-2B	100	.627	.694	91	202	100.0%	**71.9%**	75.4%	75.4%
Qwen3.5-4B	100	.789	.851	201	430	98.1%	44.0%	83.1%	84.5%
Llama-3-8B	80	.599	.619	147	440	96.4%	48.3%	66.1%	67.0%
Ministral-8B	100	.749	.797	122	372	97.4%	62.0%	78.5%	78.7%
Qwen3.5-9B	80	.791	.831	321	744	79.7%	27.6%	68.0%	83.2%
Qwen3.6-35B	2	**.822**	**.874**	265	587	90.7%	21.0%	78.0%	**84.2%**

receive smaller budgets and complete faster, reducing their own latency; simultaneously, hard queries receive proportionally larger budgets, preserving the evidence needed for multi-hop reasoning (F1: 0.750 vs C1's 0.759, only −0.9 pp). Graph-topological difficulty scoring (Phase 2) is the primary contributor to online goodput.

(B3) Backpressure adds load-proportional headroom. Adding the Phase 3→Phase 2 backpressure loop (C3→AdpK+BP) improves $\mathcal{J}$@200 by +2.3 pp (47.8% vs 45.5%) and reduces avg K from 14.7 to 12.5 (−15%) by dynamically compressing K_{eff} during congestion bursts. The improvement is modest at this load level (λ=30), consistent with RQ4's finding that backpressure contributes a load-proportional complement: its impact grows with request rate. Crucially, F1 *increases* from 0.750 to 0.755 despite lower avg K, because the difficulty-aware allocation under backpressure redistributes budget from over-provisioned easy queries to under-served hard ones.

(B4) Composite priority improves median and tail latency. DynaGraph's composite priority adds +7.8 pp $\mathcal{J}$@200 over FCFS with the same adaptive K (55.6% vs 47.8%), while reducing p50 from 173 to 143 ms (−17%) and p95 from 476 to 427 ms (−10%). The composite score combines difficulty-amplified SLO urgency and estimated prefill cost, allowing the scheduler to interleave short easy prefills between long hard ones rather than processing them in arrival order.

(B5) Brute-force compression is suboptimal. SJF with K=10 achieves the highest $\mathcal{J}$@200 (61.5%) among all configurations by aggressively shortening all prompts, surpassing DynaGraph's 55.6%. However, this comes at the cost of the lowest F1 in the table (0.742 vs DynaGraph's 0.755, −1.3 pp), because all queries—including hard multi-hop ones—receive the same compressed budget. Under relaxed SLO (τ=500 ms), DynaGraph closes the gap (77.9% vs 78.0%, within 0.1 pp), demonstrating that *selective* budget allocation preserves quality while approaching the latency of brute-force compression. This underscores the core design principle: context compression alone is insufficient; *difficulty-aware* compression coupled with serving-aware scheduling maximizes the quality–latency frontier.

Summary. The two ablations, conducted on different LLM architectures, converge on the same conclusion: (1) adaptive K is the dominant factor, providing the cost variance that all downstream optimizations depend on; (2) backpressure and composite scheduling each add incremental gains that compound under load; (3) the joint system achieves a strong quality–latency trade-off. These results provide the detailed evidence for the design claims in Section 3.2 and the closed-loop architecture in Section 4.3–4.4.

E.5 End-to-End Live Pipeline Deployment

RQ1–RQ4 and RQ6–RQ7 use pre-computed retrieval to isolate serving-layer effects. Table 19 complements RQ5 by reporting fully live end-to-end latency, where graph construction, embedding computation, reranking, and LLM inference run concurrently on the same hardware.

E2E p50 stays below 700 ms on every dataset–LLM combination. Retrieval contributes 178–352 ms depending on subgraph density (MuSiQue > HotpotQA > 2Wiki), while generation time scales with prompt length. F1 matches the pre-computed-retrieval numbers from Table 3 within 0.02, confirming that live retrieval does not erode answer quality. The pure-attention Qwen3-4B achieves lower E2E latency than the hybrid-Mamba Qwen3.5-4B on HotpotQA (395 vs 529 ms) because its shorter retrieved contexts yield faster prefill under the attention-only architecture.

E.6 Additional Scalability Results

This subsection complements Section 5.6 with additional evaluations across corpus scales, systems, and model architectures.

Cross-system corpus scalability. Figure 8(d) in the main text shows that DynaGraph's E2E latency grows sub-linearly under corpus expansion. Table 20 quantifies this advantage against ClueRAG, the strongest retrieval-optimized baseline with released scalability data, on identical HotpotQA corpus subsets (Qwen3.5-4B, TP=1, λ=3 qps). As shown there, DynaGraph's E2E p50 increases by only 32 ms (360→392 ms, +9%) as the corpus grows 7×, because FAISS-based two-hop expansion with a fixed reranker window bounds retrieval cost. In contrast, ClueRAG's iterative traversal grows close to linearly (609→2 076 ms, +241%), widening the gap from 1.7× at 1× to 5.3× at full corpus. This confirms the sub-linear scaling claim in Section 5.6 and demonstrates that DynaGraph's retrieval architecture is better suited for production-scale corpora where corpus size grows continuously.

Model scalability. As shown in Table 21, DynaGraph generalizes

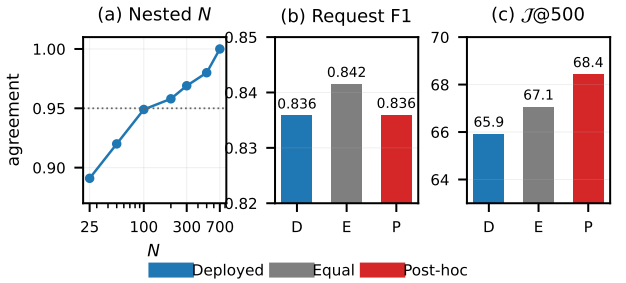


Figure 11: Weight-calibration diagnostic. (a) Agreement with the nested N=700 reference as the calibration size increases. (b–c) Request F_1 and $\mathcal{J}_\lambda$@500 under different coefficient vectors on an identical matched 5-QPS trace.

across model families and sizes without tuning and adapts effectively to different SLO regimes: (1) F_1 increases with model size within the Qwen family (0.552→0.822) while maintaining strong $\mathcal{J}$ across all models; and (2) the optimal model shifts with the latency budget—smaller models (2B) dominate under strict SLO ($\mathcal{J}$@200: 71.9%), medium models (4B) under moderate SLO, and larger models (9B/35B) under relaxed SLO. This reflects the inherent trade-off that larger models improve quality at the cost of higher latency; DynaGraph preserves this trade-off while operating in a favorable quality–latency region, as supported by the SLO-aware operating-point analysis in Figure 6. Portability beyond Qwen is confirmed by Ministral-8B ($\mathcal{J}$@500: 78.5%) and Llama-3-8B ($\mathcal{J}$@500: 66.1%), where the serving layer operates correctly (SLO@500 ≥ 96%) regardless of the underlying LLM's reasoning capability.

E.7 Static-Sweep Oracle Audit of Difficulty Signals

The earlier median-threshold "false-hard" definition conflated difficulty classification with the chosen operating point. We instead reconstruct an oracle budget K^* from the static-budget sweep and match all estimators to the same mean budget, K=12.50, over 617 unique queries. Under this audit, structural S_{diff} achieves Spearman ρ=0.260, AUROC= 0.656, and AUPRC= 0.326 for K^*>5; the adapted precomputed RAG-CSM score achieves 0.254/0.638/0.307, and the out-of-fold structural–text hybrid achieves 0.388/0.725/0.430. The paired F1 loss from K=20 to K=10 exceeds 0.1 for 4.05% of queries and 0.2 for 3.73%.

This audit is oracle-based: the RAG-CSM row uses a precomputed adaptation and does not charge selector inference to TTFT. We therefore run a separate same-stack online comparison at the highest common stable rate, 1 QPS. Table 22 replays the same seed-42 3000-request schedule with matched realized mean K and an identical request-level K histogram, while timing selector inference and feeding the live score into Phase 3.

Table 22: Online estimator comparison (2Wiki, seed 42, 1 QPS). Selector timing and the live Phase 3 path are included.

Difficulty path	F1	avg K	$\mathcal{J}$@200	$\mathcal{J}$@500	p95 (ms)	Selector p95
structural S_{diff}	.7957	12.500	63.293%	**79.267%**	**317.7**	–
live RAG-CSM	**.8004**	12.502	38.060%	75.353%	513.6	107.03
structural + live guardrail	.7957	12.500	52.240%	78.549%	395.0	89.68

Compared with live RAG-CSM, structural S_{diff} improves $\mathcal{J}$@500 by 3.914 points (95% CI [2.431, 5.977]) and reduces p95 TTFT by 195.846 ms (95% CI [−229.951, −183.661]). Its F1 difference is −0.419 points (95% CI [−1.747, 0.766]), so the supported claim is stronger strict-SLO joint quality and tail latency, not an F1 gain. Compared with the live structural+guardrail path, structural-only improves $\mathcal{J}$@200 by 11.053 points and reduces p95 TTFT by 77.257 ms, while preserving F1 within the prespecified confidence boundary.

E.8 Topology-Weight Calibration and Sensitivity

Calibration data and selection protocol. The historical procedure used the 1,000-query development workload rather than an external held-out set. We therefore evaluate calibration sufficiency through query-grouped nested validation. Across 20 splits, we fix a 300-query test fold and increase the calibration prefix from N=25 to 700, while holding the topology features and reference budgets reconstructed from the static-K sweep fixed.

Calibration-sample sufficiency. For each test fold, we compare budget-bucket assignments with those produced using the full N=700 calibration pool. Figure 11(a) shows that agreement reaches 0.958 at N=200 and 0.969 at N=300, indicating that 200–300 queries provide stable calibration with diminishing gains thereafter.

Coefficient-vector sensitivity. Figures 11(b–c) replay three coefficient vectors on the same 5-QPS schedule: the deployed vector $(0.4, 0.3, 0.3)$, the equal-weight vector $(1/3, 1/3, 1/3)$, and the historical post-hoc vector selected from the full N=700 calibration pool using the same grid search. Across these vectors, request F_1 varies by only 0.56 percentage points, mean K ranges from 13.46 to 13.76, and $\mathcal{J}_\lambda$@500 ranges from 65.92 to 68.43. This 5-QPS setting is a calibration diagnostic rather than the 30-QPS main protocol, and the post-hoc vector was not deployed. The stability arises because $S_{\text{diff}}(q) = \mathbf{w}^\top \tilde{\mathbf{f}}_q$ provides only a coarse budget ordering, while backpressure further contracts residual allocation differences under load. Thus, the topology score requires no trained estimator and remains stable under moderate changes in both the calibration sample and coefficient vector, without sensitive retuning.

E.9 Energy Efficiency and Token-Level Streaming

This subsection evaluates GPU energy under adaptive and static evidence budgets and examines whether adaptive budgeting affects token-level streaming cadence. We report serving-run GPU-energy measurements, a matched-budget attribution, and token-level inter-token latency.

GPU energy measurements. Following prior work on LLM-inference energy measurements [40, 57], we measure GPU energy for the runs underlying Table 3. On the official 30-QPS trace, relative to static K=20, DynaGraph reduces GPU energy per request by 24.7% (94.09 vs. 124.96 J) and per 1K output tokens by 15.6%, while improving $\mathcal{J}$@500 from 71.19% to 79.45%. These measurements exclude host energy and monetary cost.

Matched-budget energy attribution. To separate the effect of adap-

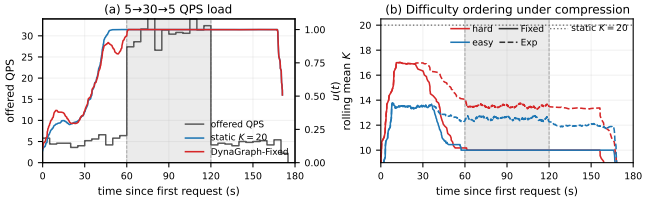


Figure 13: Load-and-difficulty burst. The shaded interval denotes the 30-QPS peak with 70% hard-query arrivals. (a) Offered and served QPS. (b) Mean budgets for hard/easy queries under different budget controllers.

tive control from that of shorter contexts, we run static $K \in \{10, 13, 20\}$ and DynaGraph three times each at 5 QPS, using identical request schedules, fresh servers, warmup, idle-power subtraction, and dual-GPU NVML sampling. As shown in Table 23, DynaGraph uses 9.35% less GPU energy per request than static K=20. However, DynaGraph uses 3.09% more energy than the approximately budget-matched static K=13. Thus, the measured reduction relative to static K=20 primarily comes from shorter admitted contexts; the matched-budget comparison does not isolate additional energy savings attributable to the controller itself.

Table 23: Matched-budget GPU-energy attribution (5 QPS; three-run means). Static K=13 approximately matches DynaGraph's mean evidence budget.

Configuration	Mean K	GPU energy/req. (J)	Reduction vs. K=20
Static K=20	20.00	110.63	–
Static K=10	10.00	95.38	13.8%
Static K=13	13.00	97.28	12.1%
DynaGraph	12.93	100.29	9.35%

Token-level inter-token latency. We instrument vLLM before SSE coalescing and record a monotonic timestamp for each generated token. Across 3,000 requests per policy, relative to static K=20, DynaGraph preserves the median inter-token latency (ITL; 25.00 vs. 25.33 ms), reduces p95/p99 ITL from 298.55/484.34 to 203.46/315.33 ms, and lowers the rate of gaps above 200 ms from 7.95% to 5.36%. The 50- and 100-ms exceedance rates increase slightly; thus, the improvement is concentrated in severe decode stalls rather than in every token gap (Figure 12).

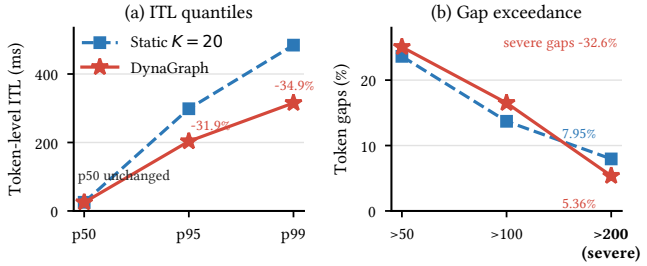


Figure 12: Token-level streaming cadence. Relative to static K=20, (a) DynaGraph preserves p50 cadence and reduces p95/p99 ITL by 31.8%/34.9%; (b) the 50/100-ms exceedance rates increase slightly, whereas gaps above 200 ms decrease by 32.6%.

E.10 Load-and-Difficulty Bursts and Controller Recovery

This subsection evaluates DynaGraph under a traffic spike coinciding with hard-query arrivals and its subsequent recovery. We first compare budget controllers and track end-to-end budget contraction during the burst, then report complementary post-spike recovery diagnostics.

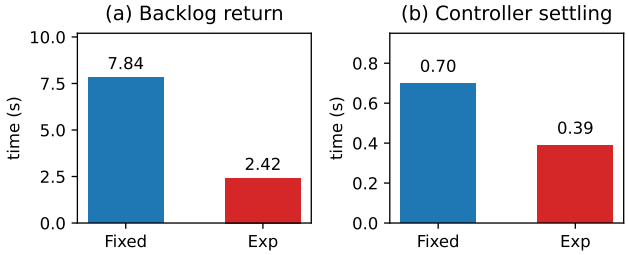


Figure 14: Post-spike recovery diagnostics under configuration-specific traces. (a) Time for the backlog to return to its pre-burst level; (b) smoothed-controller settling time. No post-peak threshold recrossing or controller overshoot is observed for either configuration.

Load-and-difficulty bursts. High arrival rates can coincide with evidence-demanding queries, jointly increasing queue pressure and per-request prefill cost. To evaluate this setting, we use a $5 \to 30 \to 5$-QPS burst with 70% hard-query arrivals at the peak. We compare static K=20 with two decay variants: DynaGraph-Exp, the default exponential controller, and DynaGraph-Fixed, an aggressive constant-ceiling variant that compresses budgets toward $K_{\min}$ under peak load. As shown in Figure 13, static K=20 yields hard-query p95 TTFT of 848 ms and $\mathcal{J}$@500 of 70.3%. DynaGraph-Exp improves these to 668 ms and 78.4% with a 0.57-point F1 loss, while DynaGraph-Fixed further improves them to 435 ms and 86.9%, with a larger 1.58-point loss. Exponential decay retains higher hard/easy budgets (13.51/12.55), preserving more evidence and difficulty differentiation, whereas the fixed variant compresses both strata more aggressively. These results show that the feedback loop supports different quality–latency operating points: exponential decay preserves more evidence, whereas the fixed controller trades additional quality for stronger strict-SLO protection.

Traffic-spike controller response. We track utilization $u(t)$, the effective evidence-budget ceiling $K_{\text{eff}}(t)$, and per-request TTFT over a $5 \to 30 \to 5$-QPS transition, with 70% hard-query arrivals during the peak. Mean utilization rises from 0.054 before the peak to 0.787 during the peak and returns to 0.060 afterward, while mean $K_{\text{eff}}(t)$ contracts from 20.00 to 13.27 and recovers to 19.96. Relative to static K=20, DynaGraph reduces burst-phase p95 TTFT from 523.9 to 413.9 ms and improves SLO@500 from 92.95% to 97.99% and $\mathcal{J}$@500 from 77.51% to 81.28%. These results link the traffic spike to the controller's budget contraction and subsequent release.

Post-spike recovery diagnostics. To further characterize post-spike recovery, we evaluate DynaGraph-Fixed and DynaGraph-Exp un-

der configuration-specific traces scaled to their independently profiled serving capacities. DynaGraph-Fixed uses a $1.2 \rightarrow 2.5 \rightarrow 1.2$-QPS trace, whereas DynaGraph-Exp uses a $0.6 \rightarrow 1.25 \rightarrow 0.6$-QPS trace. Figure 14 reports the observed backlog-return and smoothed-controller settling times.

Under these configuration-specific traces, DynaGraph-Fixed and DynaGraph-Exp return the backlog to its pre-burst level in 7.838 and 2.423 s and settle in 0.701 and 0.390 s, respectively. No post-peak threshold recrossing or controller overshoot is observed in either trace. These results provide complementary evidence that both configurations recover after the traffic spike.